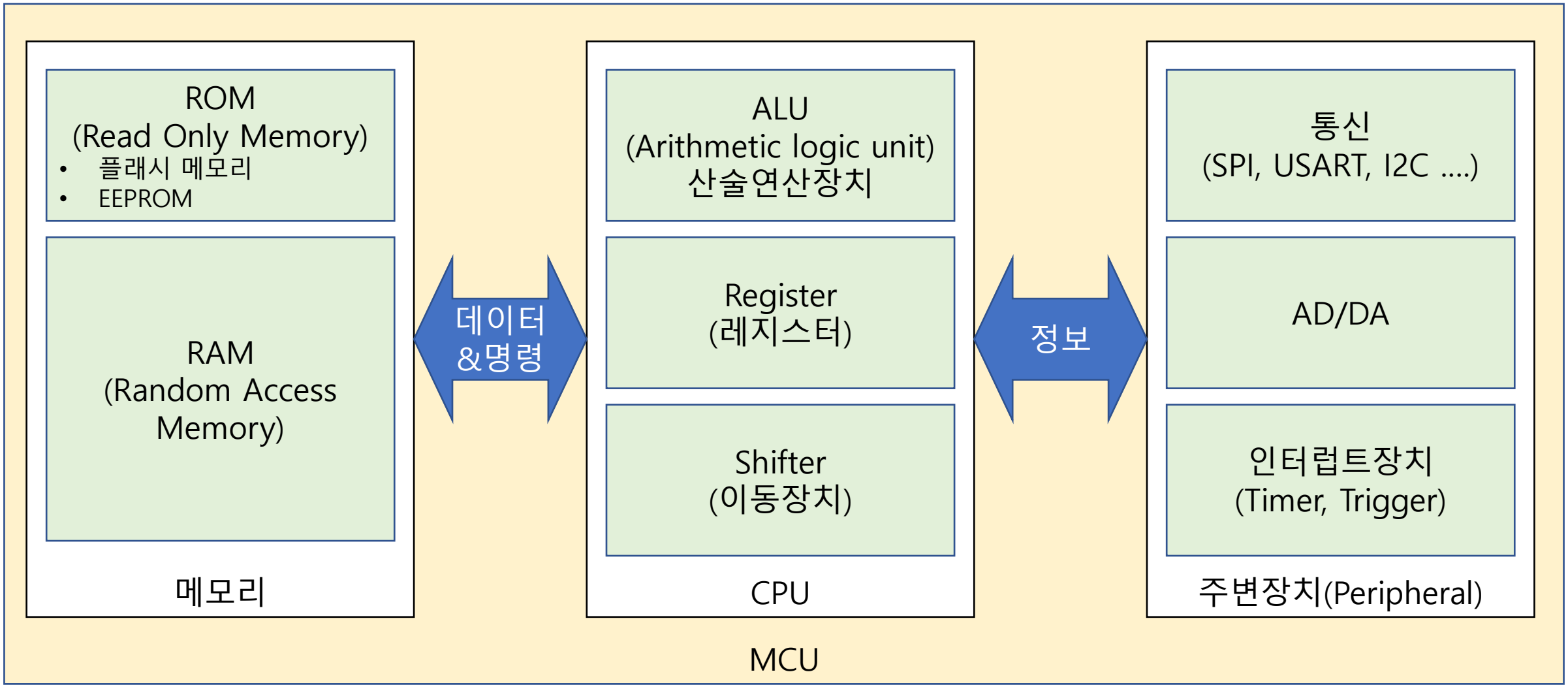
마이크로프로세서 이론

마이크로프로세서 종합 설계. 3주차.

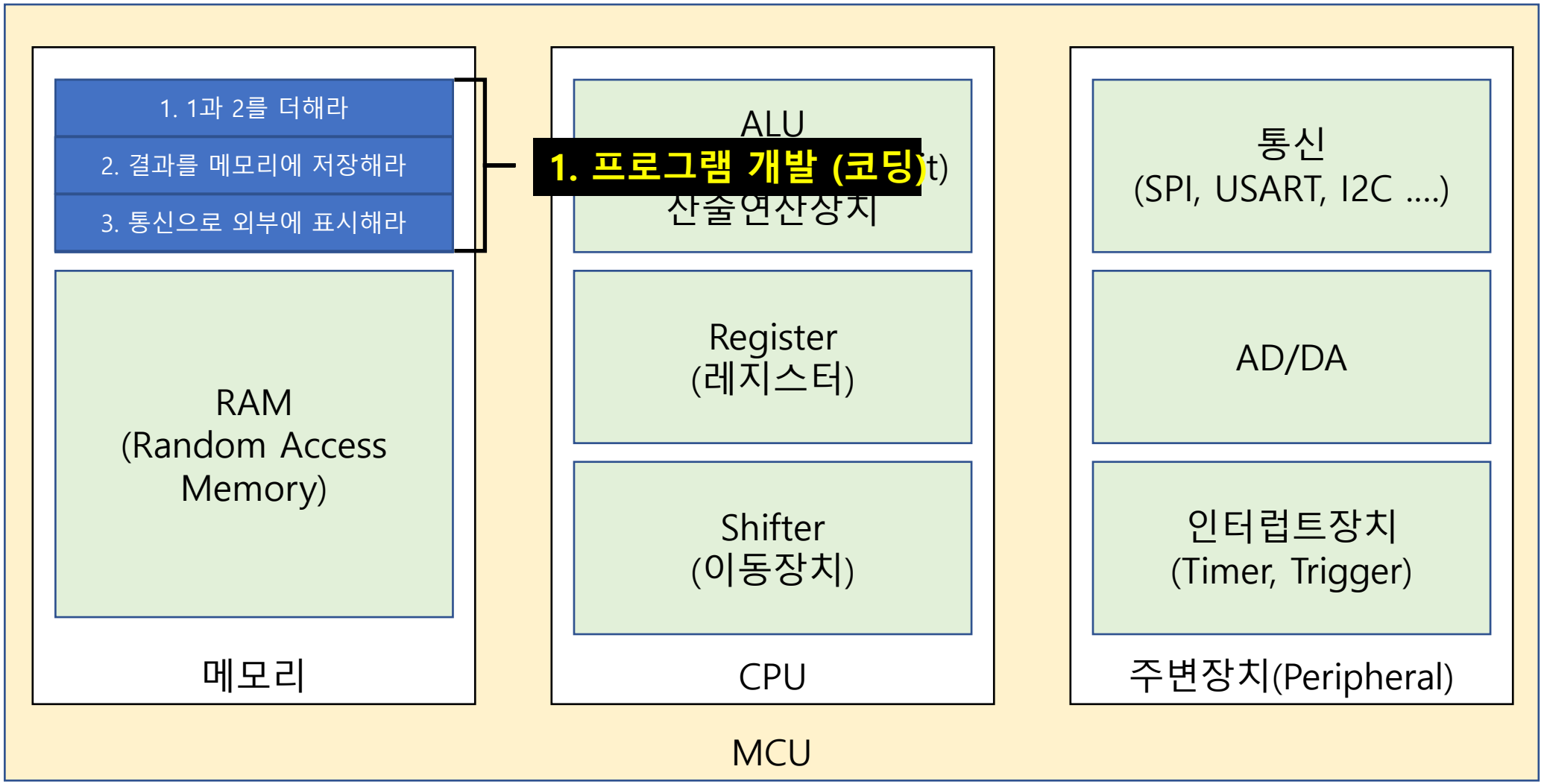
오늘의 목표

1. 마이크로프로세서의 기본 동작을 이해한다.
2. 아두이노를 첫 실험(**시리얼통신**)
3. **디지털 입출력 실험**을 통한 **레지스터의 이해**

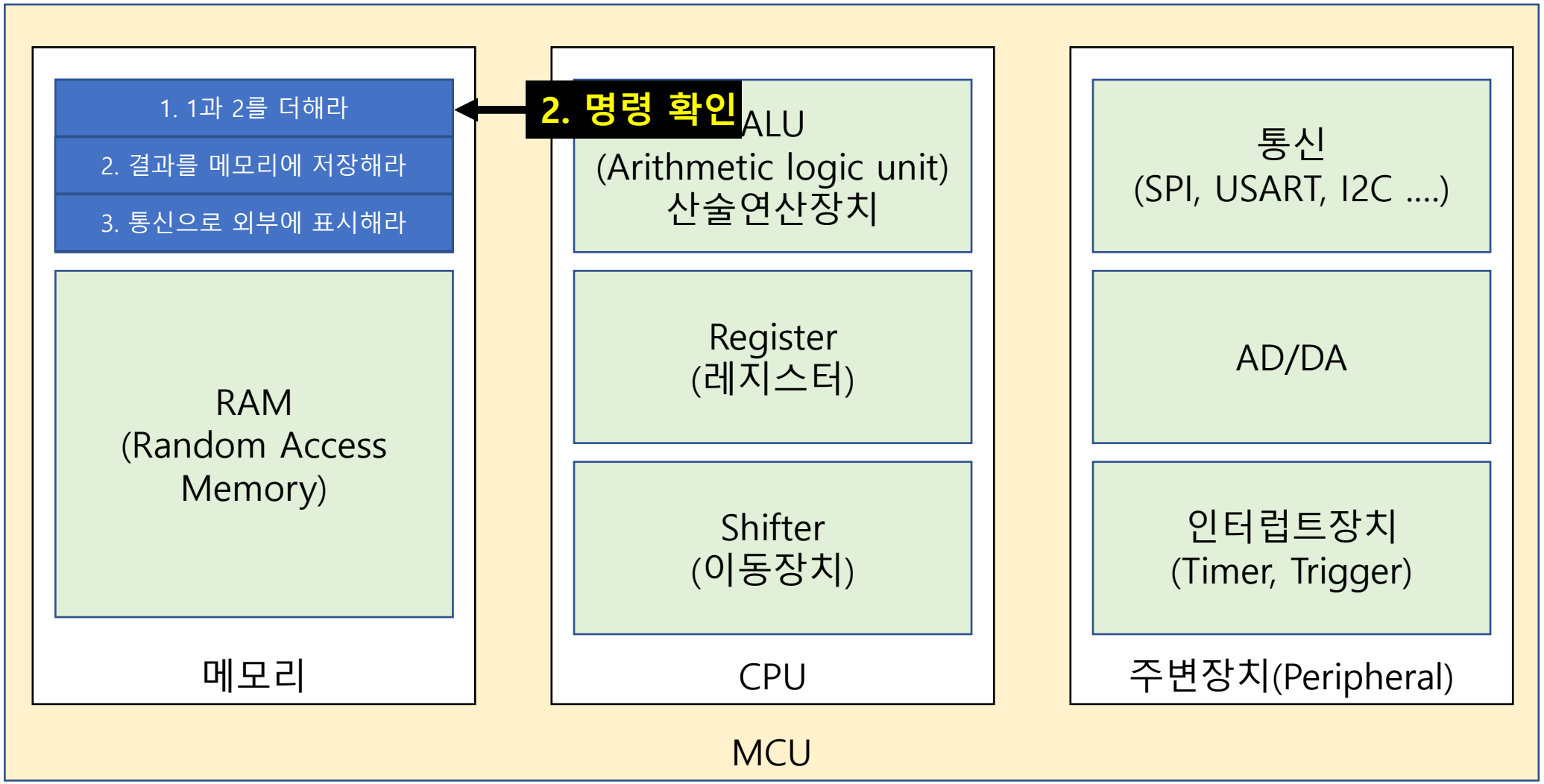
마이크로프로세서의 기본 구성



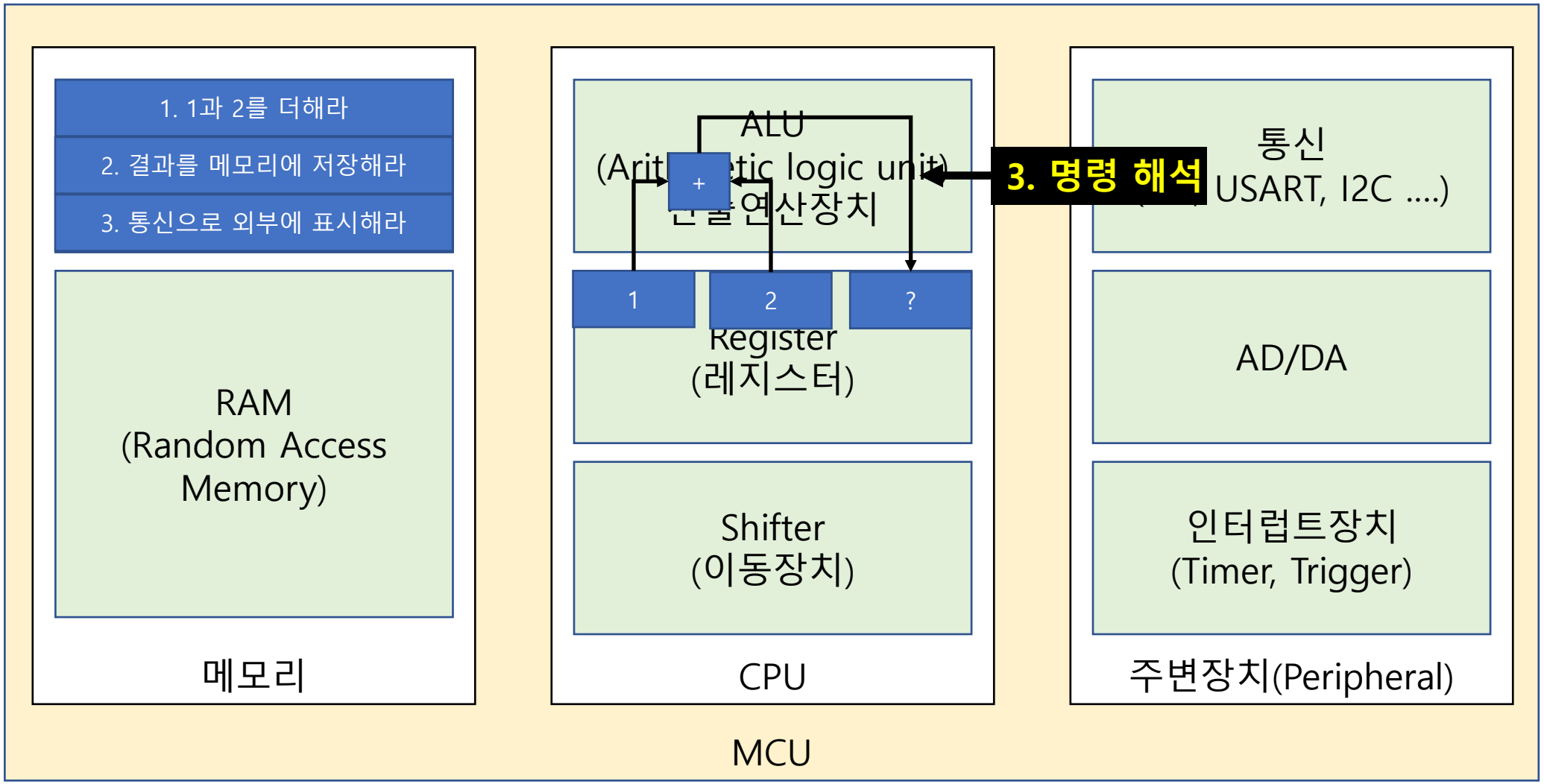
마이크로프로세서는 어떻게 명령을 수행 할까?



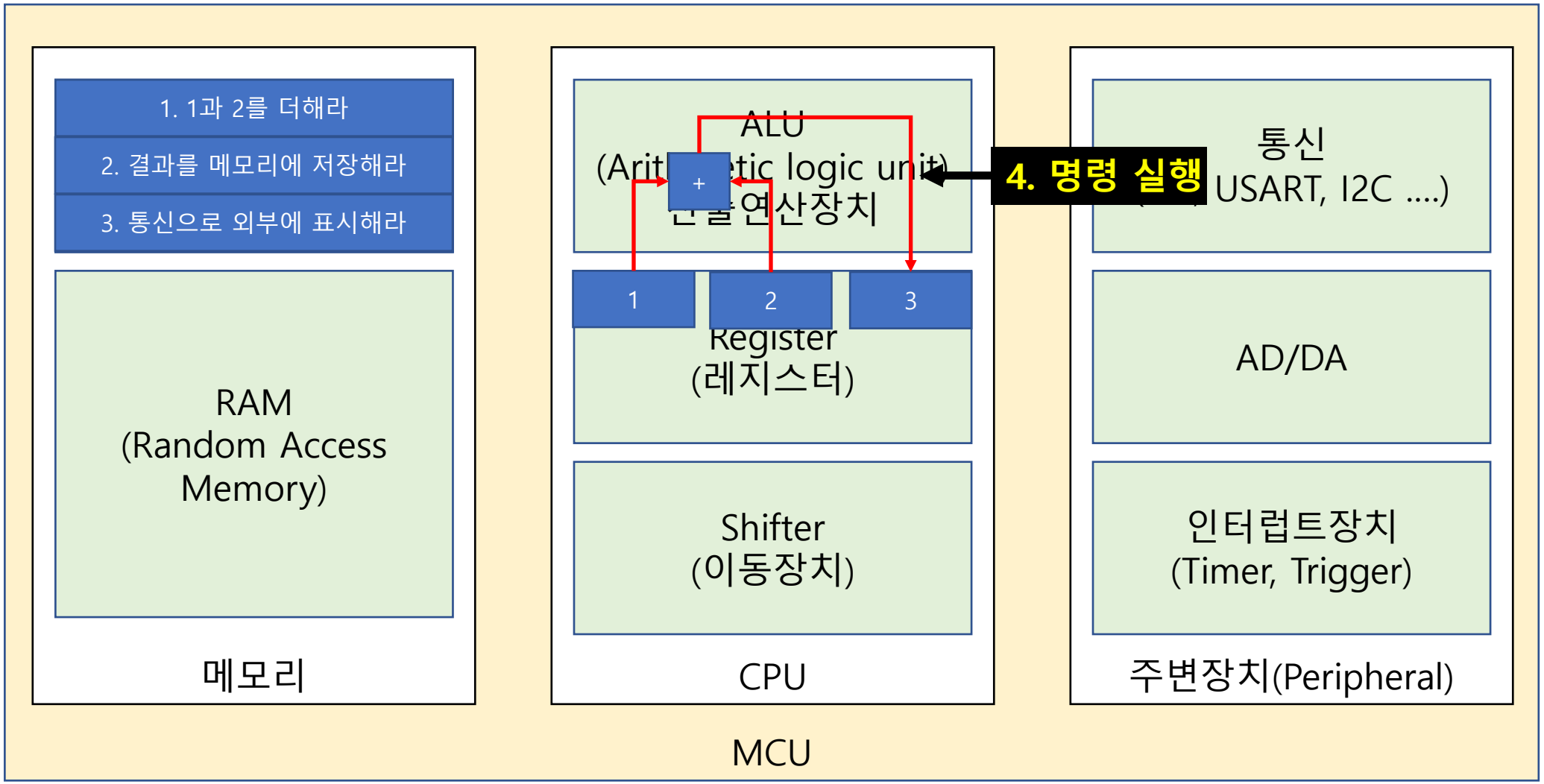
마이크로프로세서는 어떻게 명령을 수행 할까?



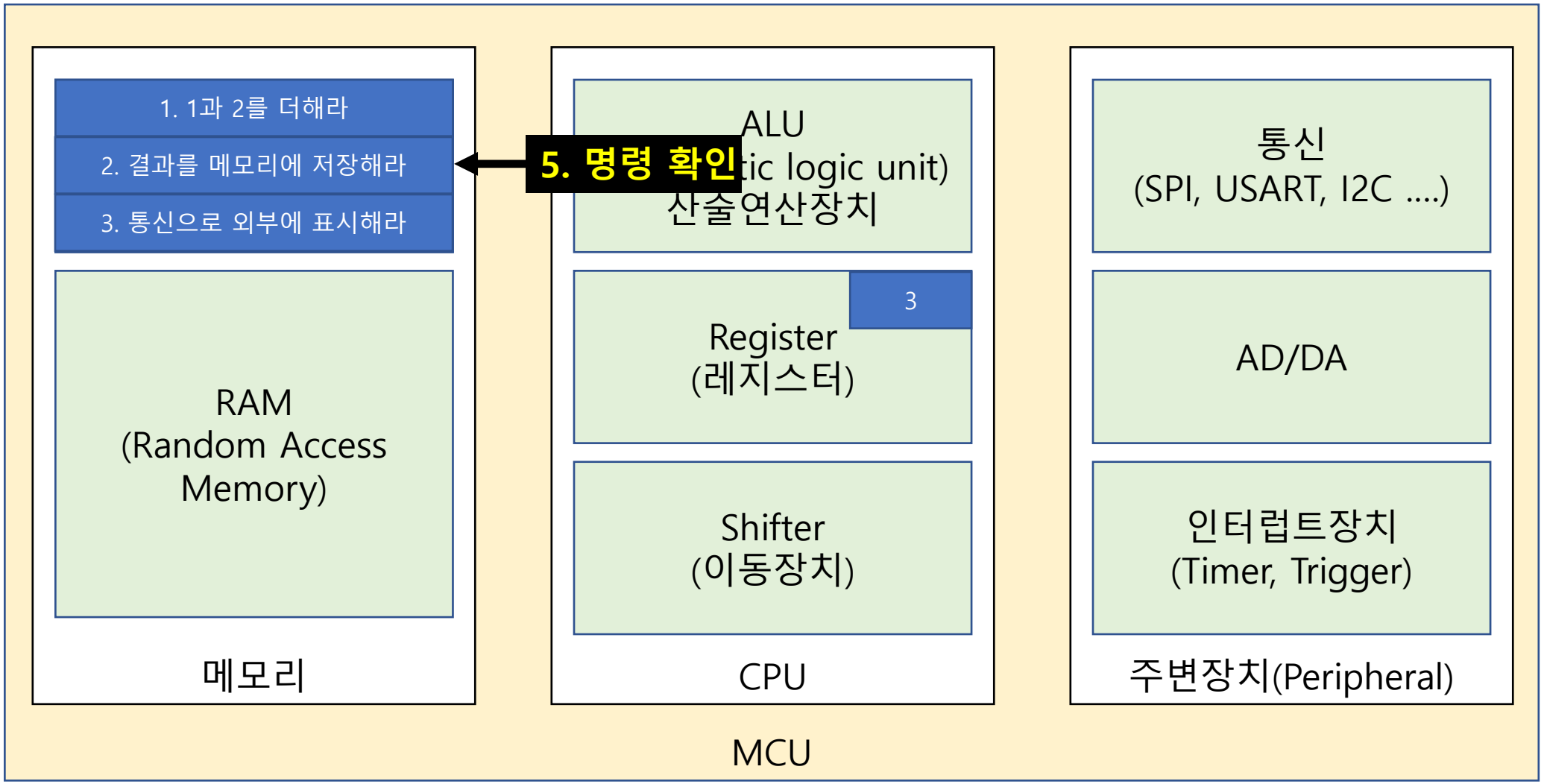
마이크로프로세서는 어떻게 명령을 수행 할까?



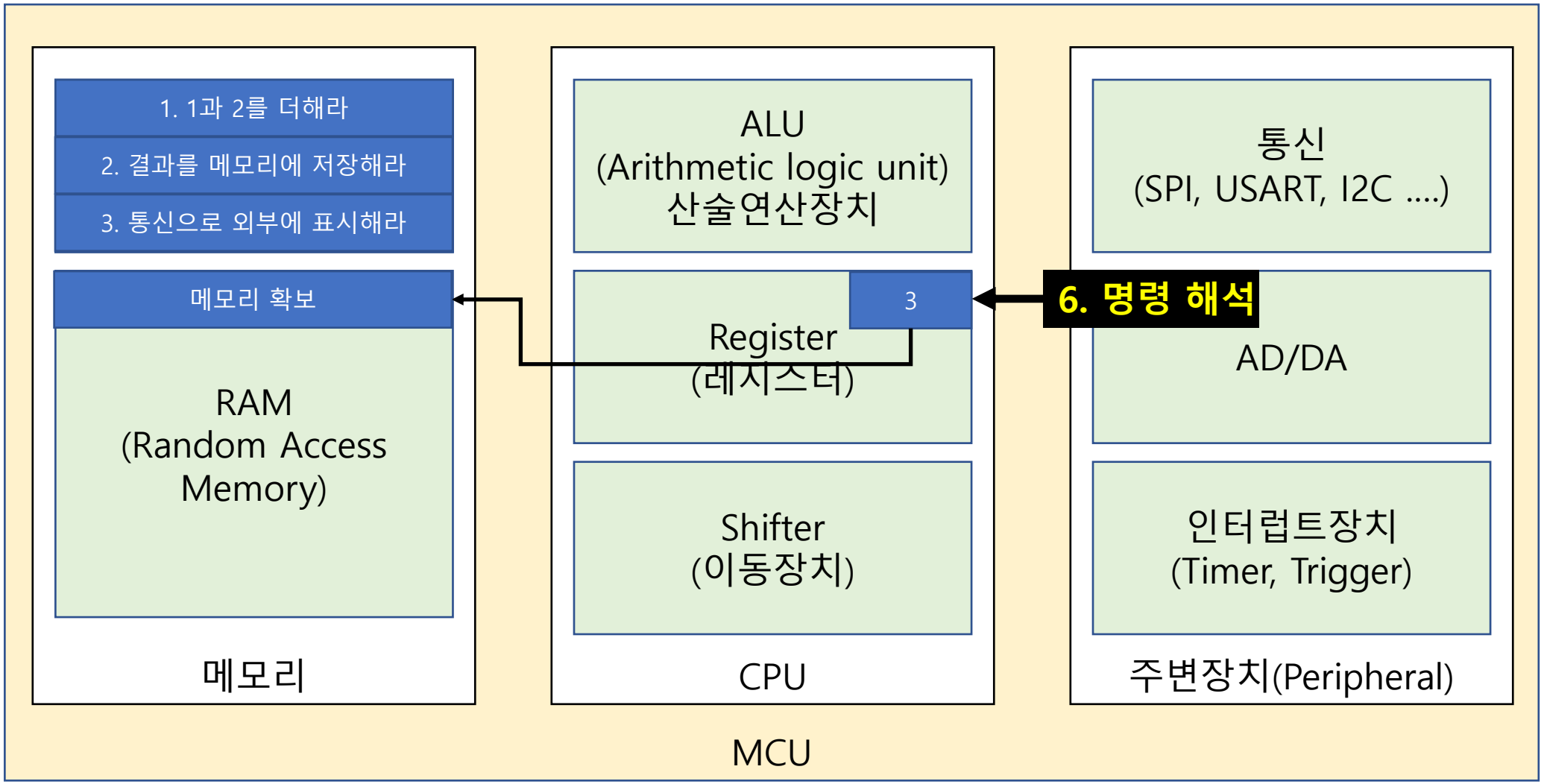
마이크로프로세서는 어떻게 명령을 수행 할까?



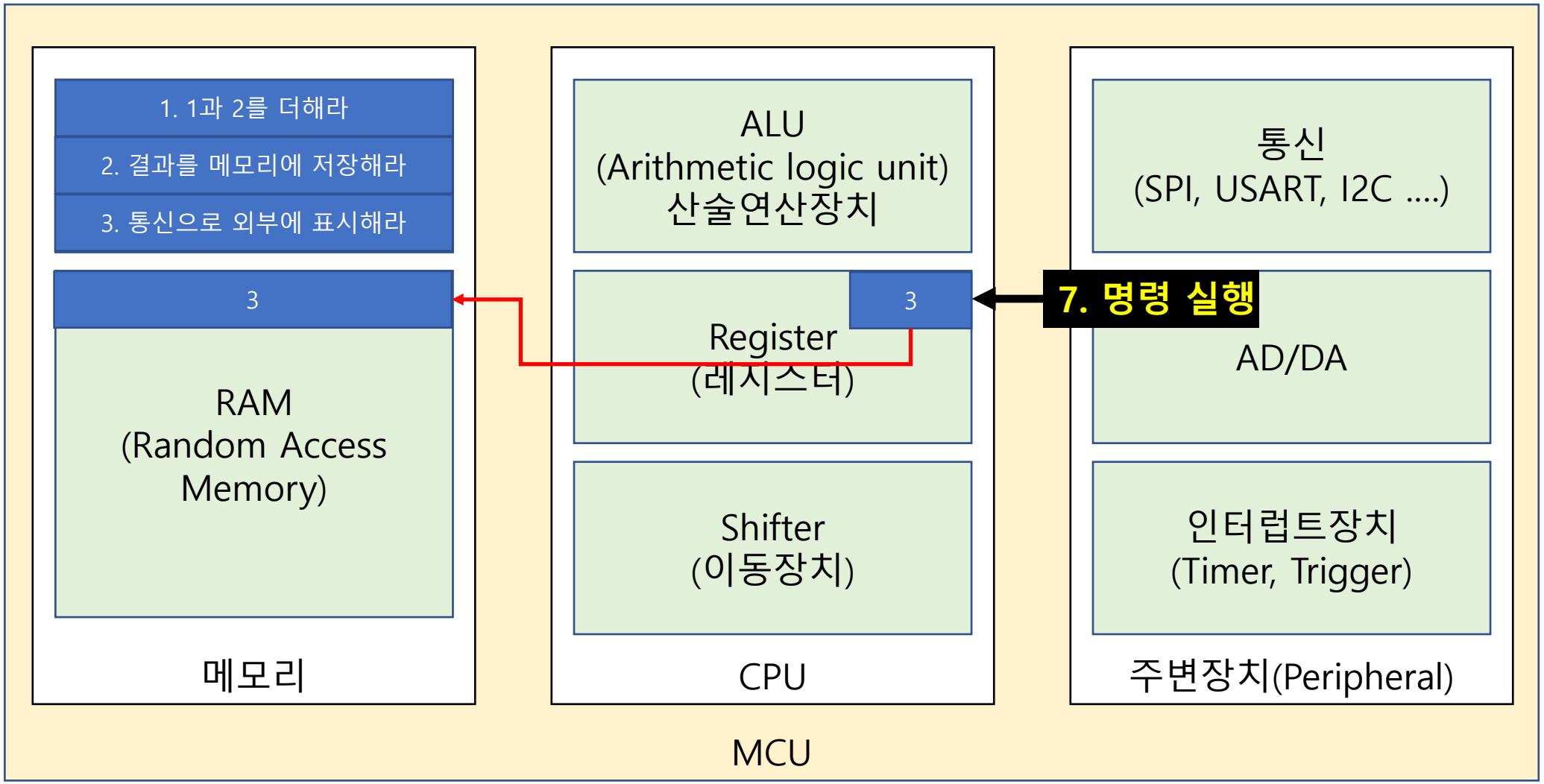
마이크로프로세서는 어떻게 명령을 수행 할까?



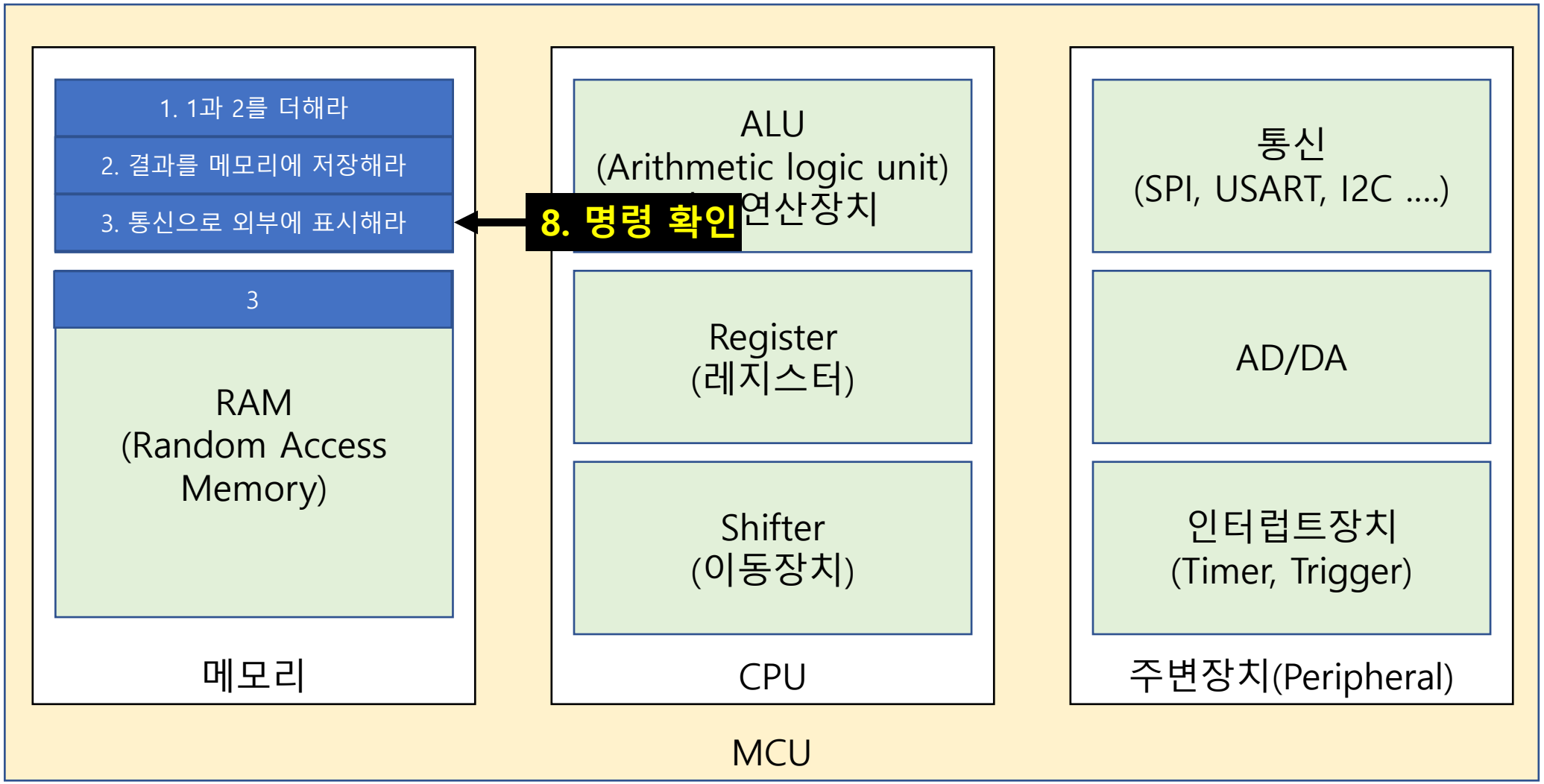
마이크로프로세서는 어떻게 명령을 수행 할까?



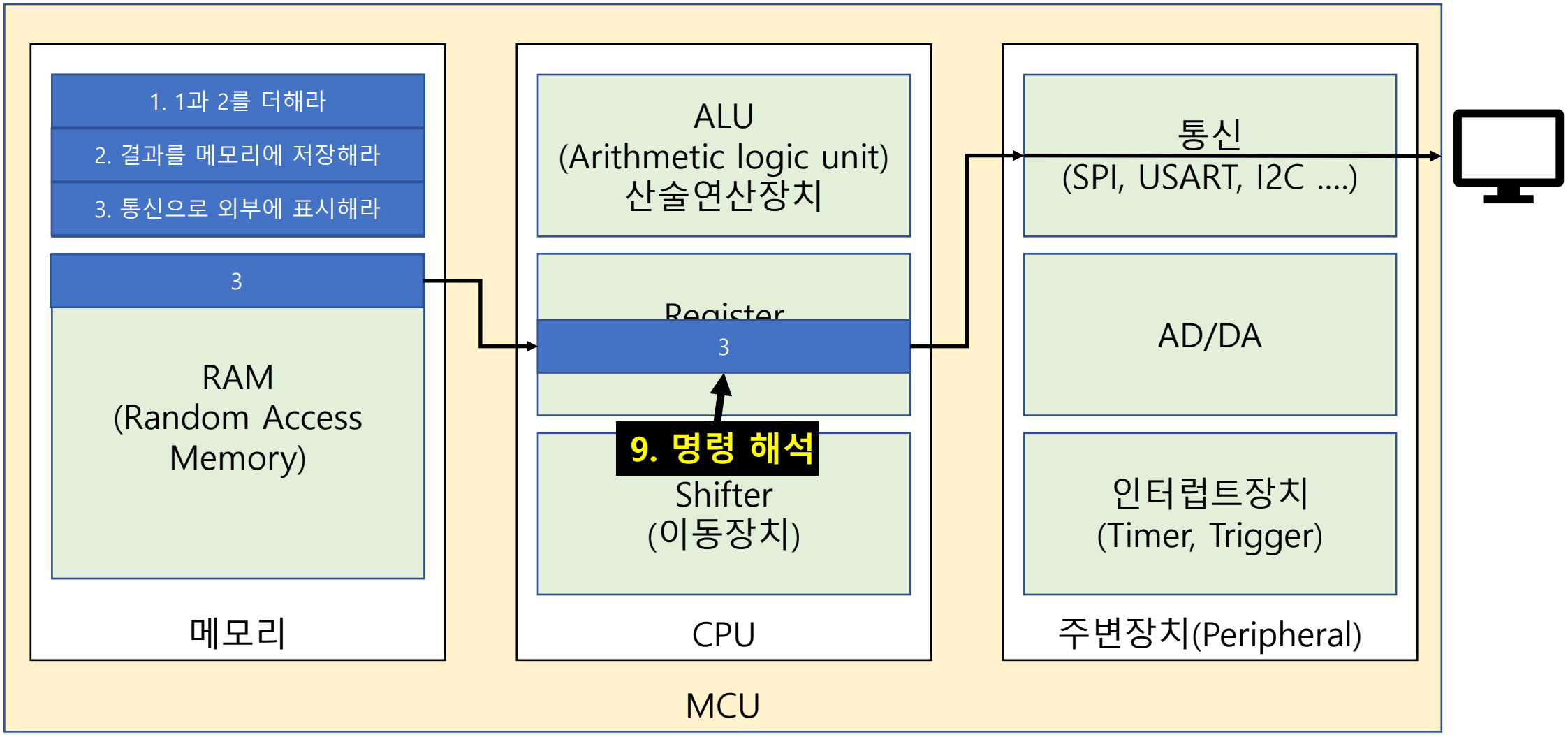
마이크로프로세서는 어떻게 명령을 수행 할까?



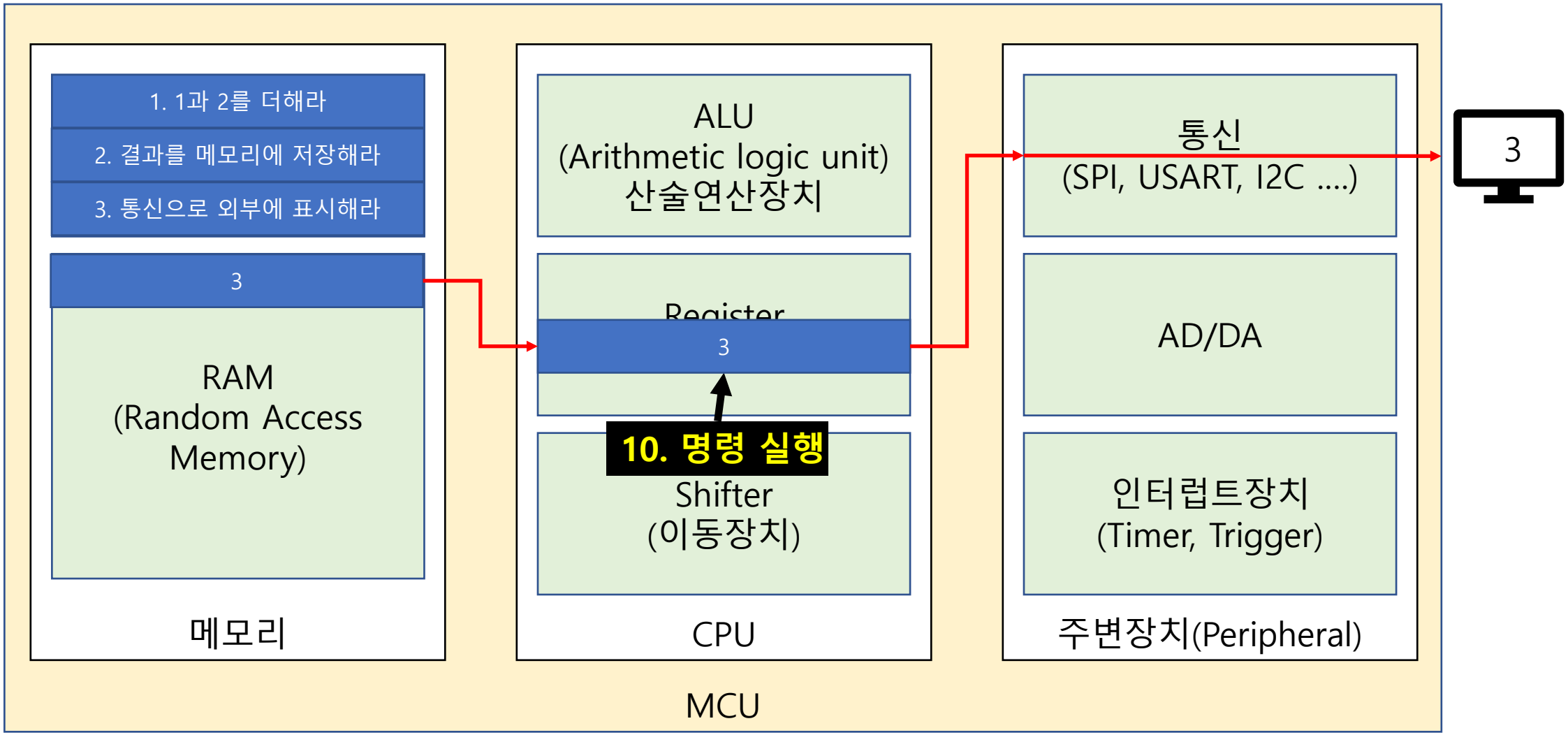
마이크로프로세서는 어떻게 명령을 수행 할까?



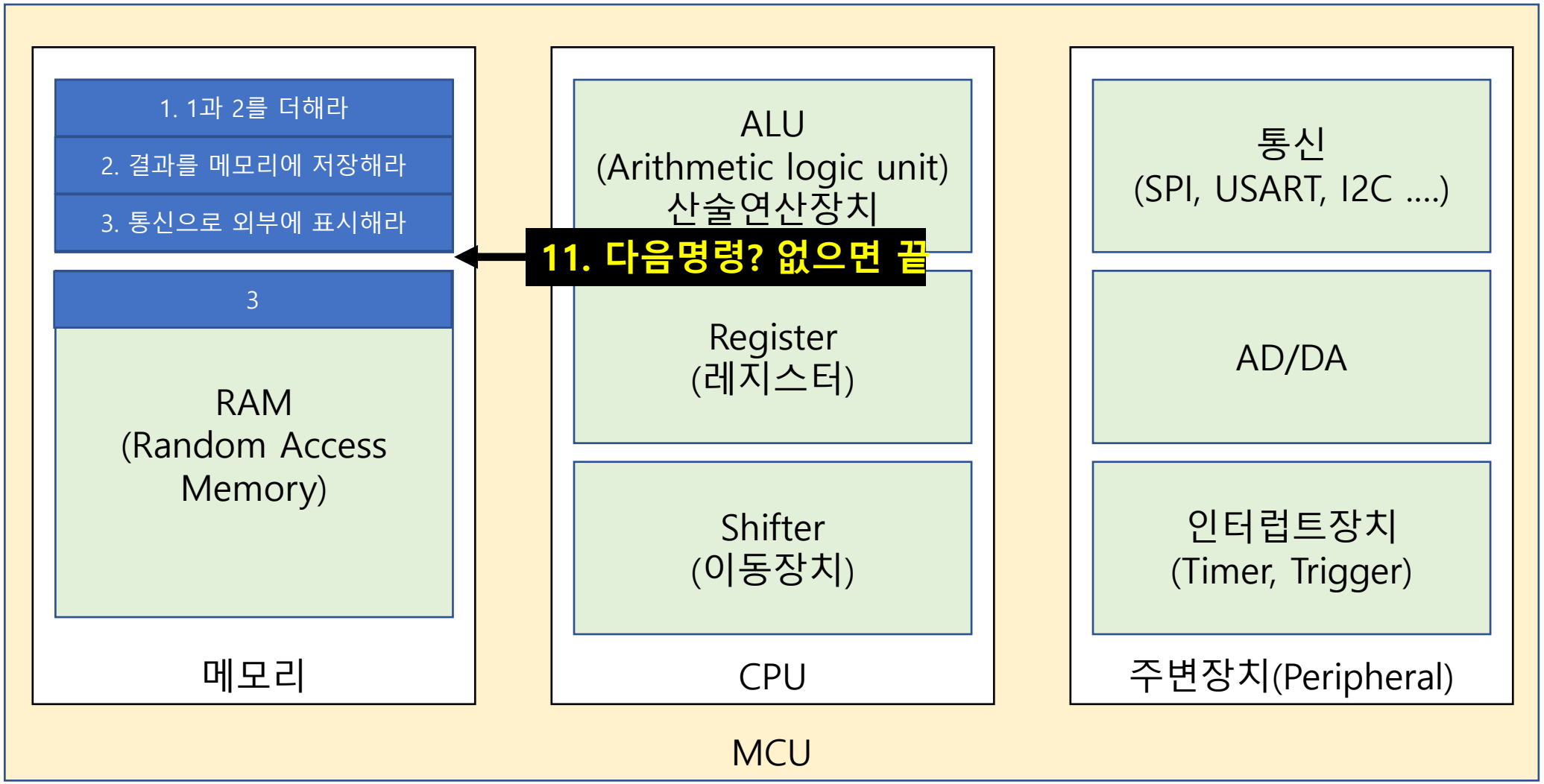
마이크로프로세서는 어떻게 명령을 수행 할까?



마이크로프로세서는 어떻게 명령을 수행 할까?



마이크로프로세서는 어떻게 명령을 수행 할까?



ATMEGA328P

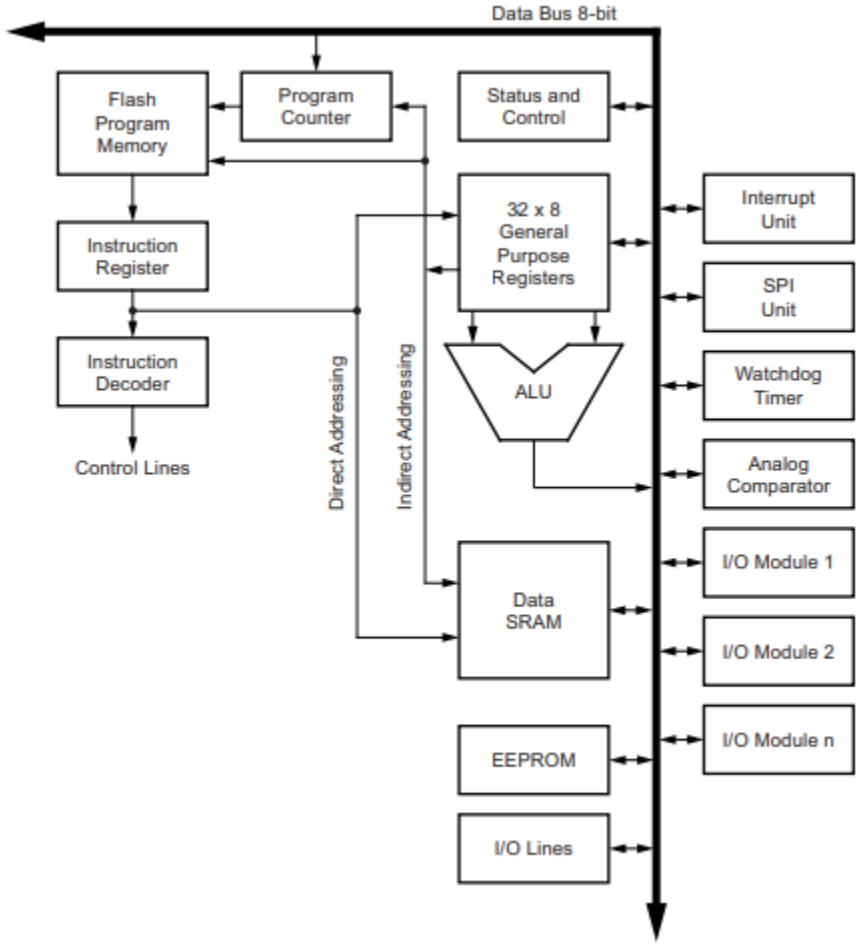
- RISC(Reduced Instruction Set Computer) 구조
  - 적은 수의 명령어로 명령어 집합을 구성하며 복잡한 명령은 명령어를 조합하여 사용
  - Single clock cycle execution동작이 가능한 131개의 명령어로 구성되어 있음
  - 32x8 general register

- 내장 메모리
  - 32kbyte의 flash memory(프로그램 메모리)
  - 1kbyte 크기의 EEPROM
  - 2kbyte 크기의 SRAM

- 주변장치(Peripheral)
  - 2개의 8비트 Timer/Counters, 1개의 16비트 Timer/Counters
  - 6개의 PWM 채널, 8채널 16비트 ADC
  - USART, SPI, I2C, Watchdog
  - 아날로그 비교기
  - 외부 인터럽트
  - 23개의 IO

데이타시트 : https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf

ATMEGA328P 기본 구조



데이타시트 : https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf

ATMEGA328P 기본 구조

프로그램카운터 (Program Counter : PC)

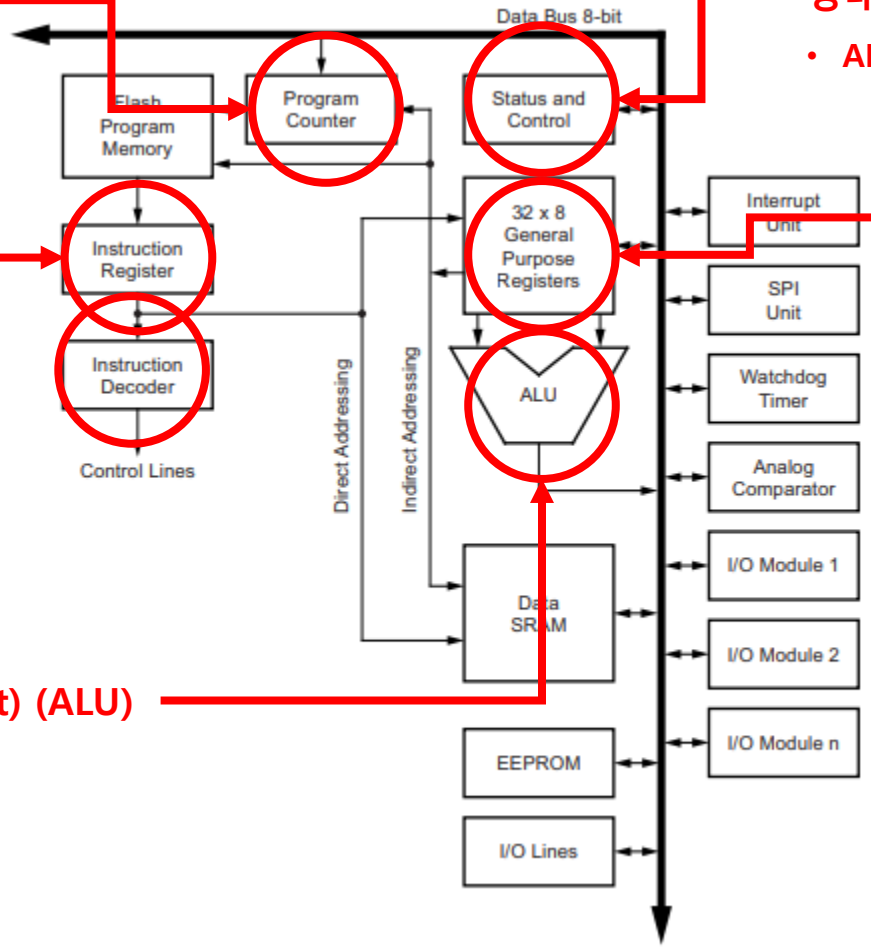
- 내가 실행시킨 명령어의 주소를 가리키는 역할

명령어 레지스터

- 실행 해야 하는 명령어를 저장

산술 논리 장치 (Arithmetic Logical Unit) (ALU)

- 4칙연산
- 논리(AND, OR, NOT)
- 비트연산



상태 레지스터 (Status Register : SREG)

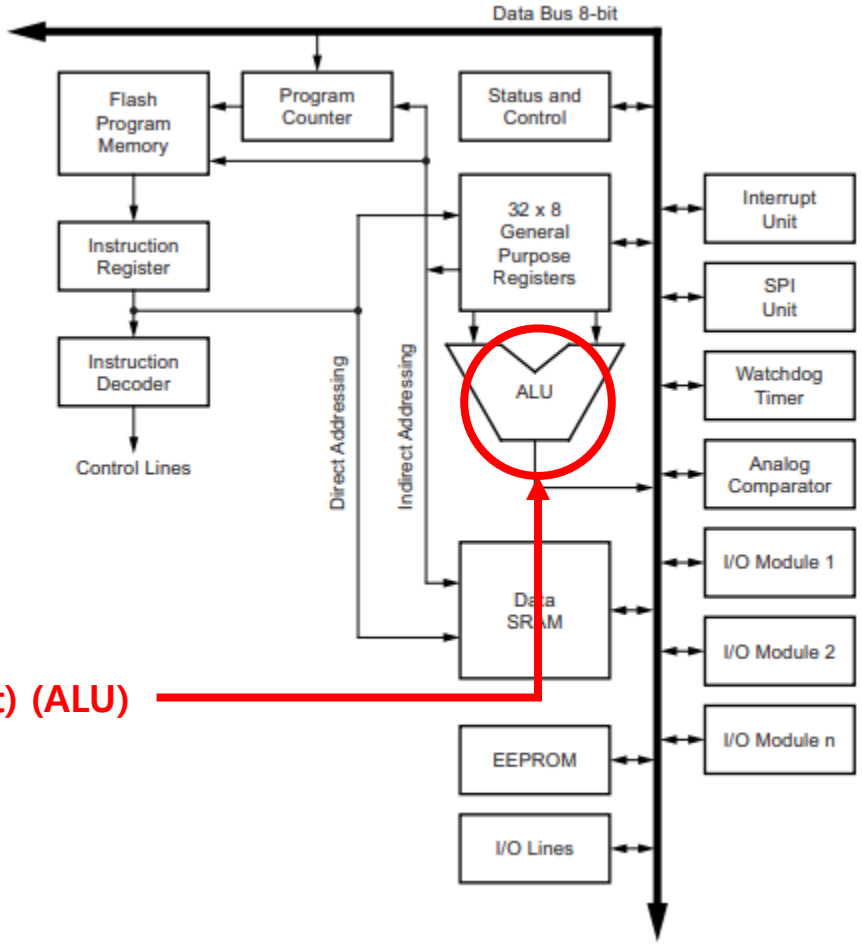
- ALU가 가장 최근에 실행한 연산 명령의 결과와 상태를 표시

범용 레지스터 (General Purpose Register)

- 프로그램 수행 중에 중간 결과나 데이터를 일시적으로 저장하는데 사용

데이타시트 : https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf

ATMEGA328P 기본 구조

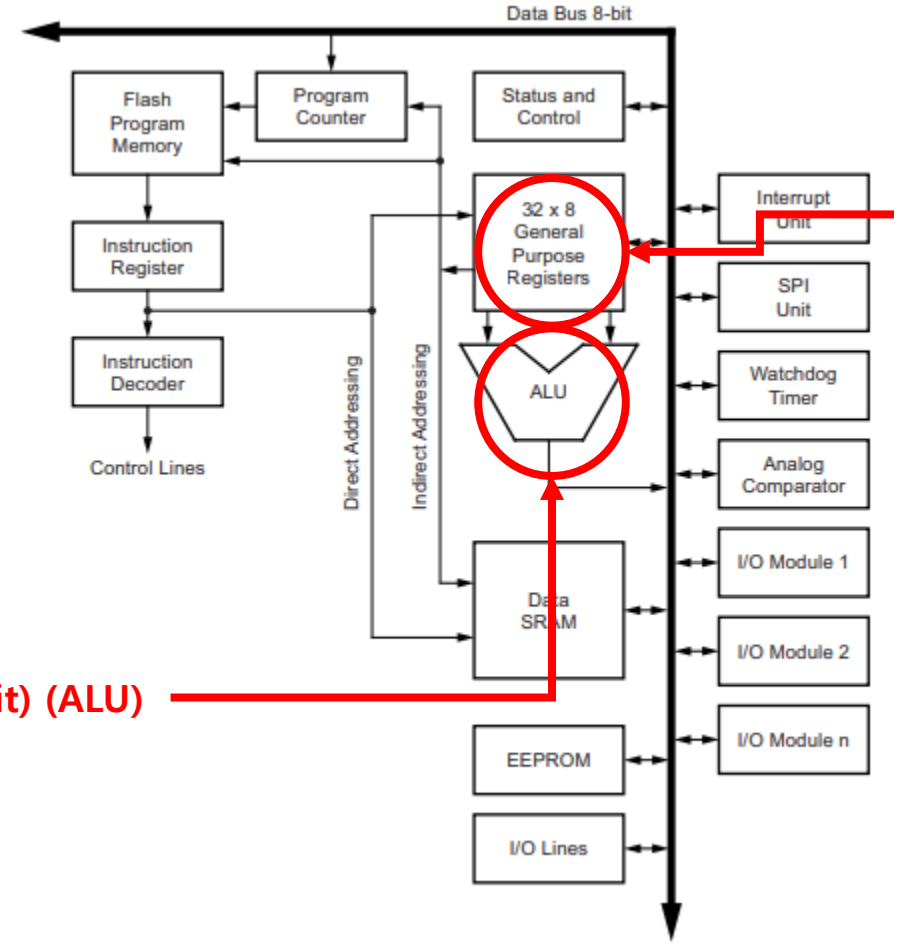


산술 논리 장치 (Arithmetic Logical Unit) (ALU)

- **4칙연산**
- **논리(AND, OR, NOT)**
- **비트연산**

데이타시트 : https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf

ATMEGA328P 기본 구조



범용 레지스터 (General Purpose Register)

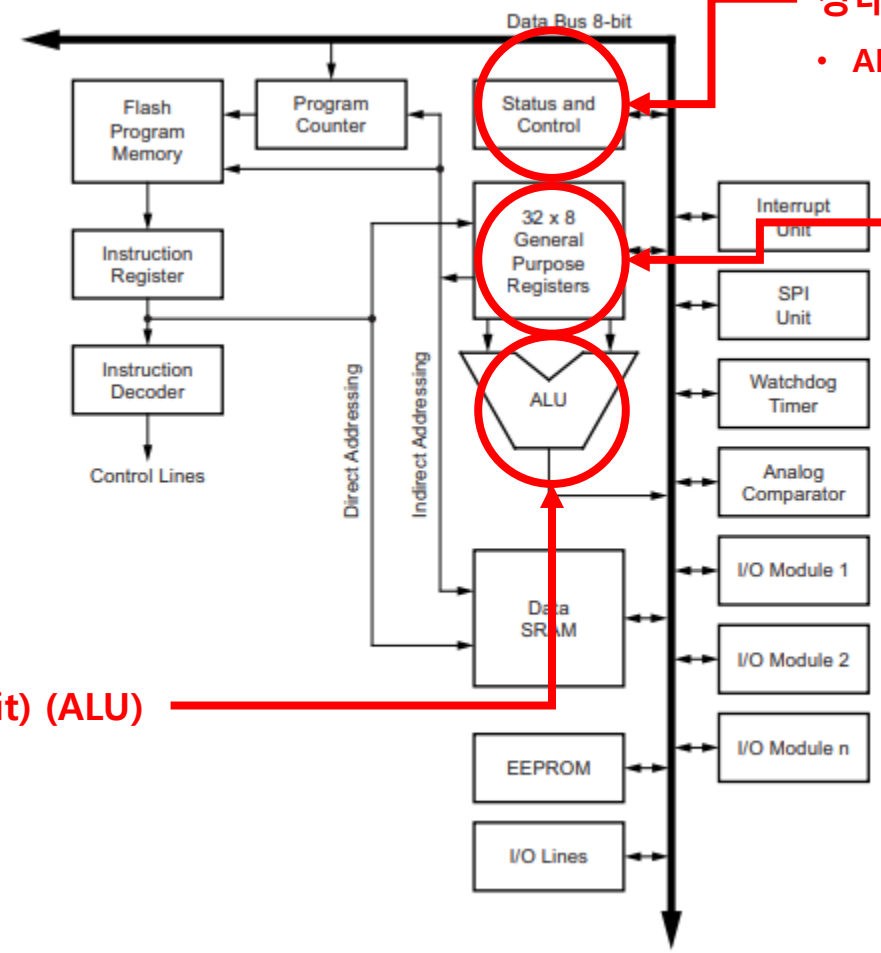
- 프로그램 수행 중에 중간 결과나 데이터를 일시적으로 저장하는데 사용

산술 논리 장치 (Arithmetic Logical Unit) (ALU)

- 4칙연산
- 논리(AND, OR, NOT)
- 비트연산

데이타시트 : https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf

ATMEGA328P 기본 구조



상태 레지스터 (Status Register : SREG)

- **ALU가 가장 최근에 실행한 연산 명령의 결과와 상태를 표시**

범용 레지스터 (General Purpose Register)

- **프로그램 수행 중에 중간 결과나 데이터를 일시적으로 저장하는데 사용**

산술 논리 장치 (Arithmetic Logical Unit) (ALU)

- **4칙연산**
- **논리(AND, OR, NOT)**
- **비트연산**

데이타시트 : https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf

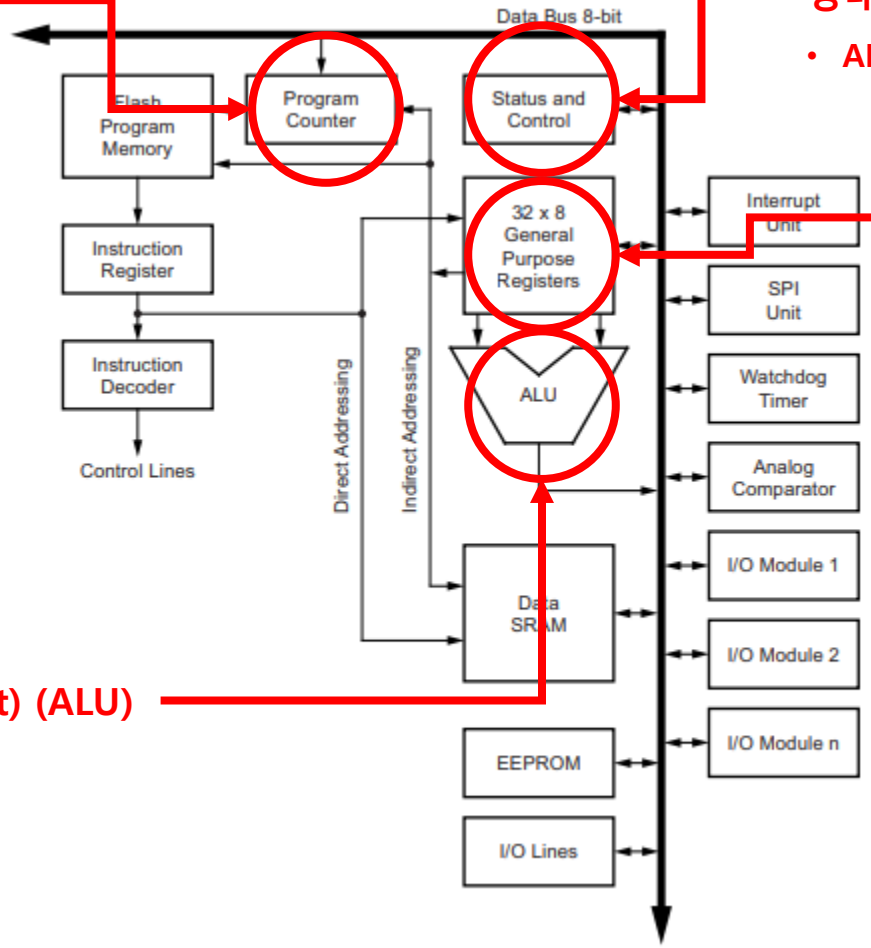
ATMEGA328P 기본 구조

프로그램카운터 (Program Counter : PC)

- 내가 실행시킨 명령어의 주소를 가리키는 역할

상태 레지스터 (Status Register : SREG)

- ALU가 가장 최근에 실행한 연산 명령의 결과와 상태를 표시



범용 레지스터 (General Purpose Register)

- 프로그램 수행 중에 중간 결과나 데이터를 일시적으로 저장하는데 사용

산술 논리 장치 (Arithmetic Logical Unit) (ALU)

- 4칙연산
- 논리(AND, OR, NOT)
- 비트연산

데이타시트 : https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf

ATMEGA328P 기본 구조

프로그램카운터 (Program Counter : PC)

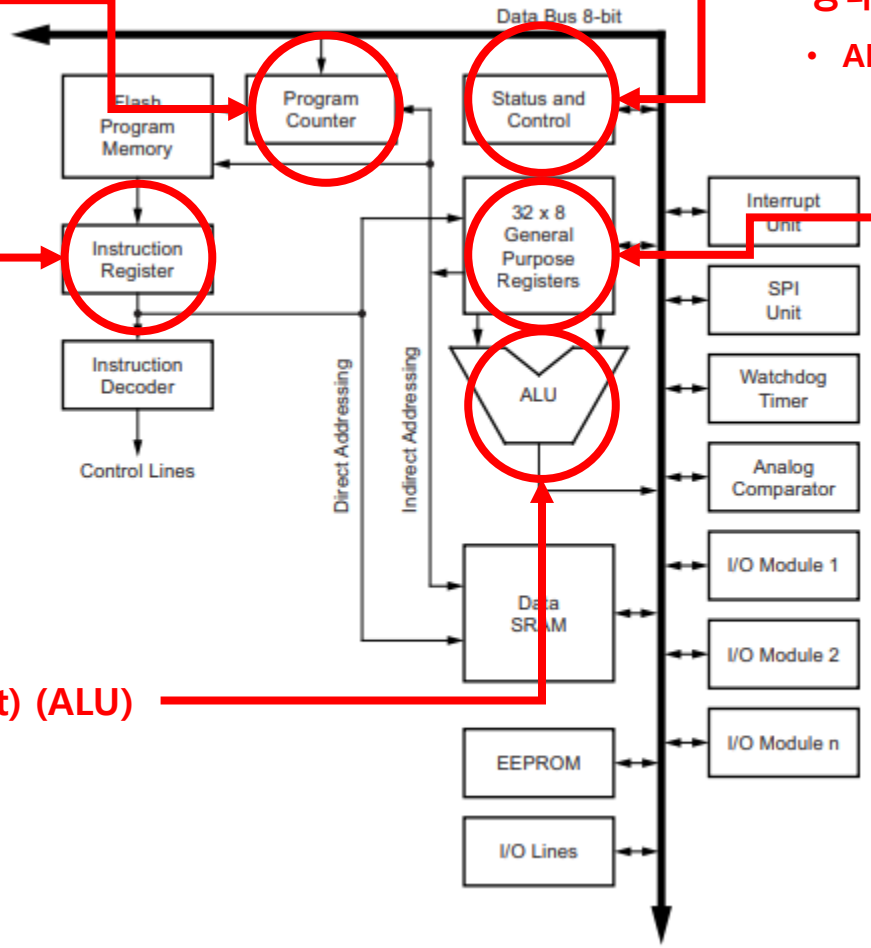
- 내가 실행시킨 명령어의 주소를 가리키는 역할

명령어 레지스터

- 실행 해야 하는 명령어를 저장

산술 논리 장치 (Arithmetic Logical Unit) (ALU)

- 4칙연산
- 논리(AND, OR, NOT)
- 비트연산



상태 레지스터 (Status Register : SREG)

- ALU가 가장 최근에 실행한 연산 명령의 결과와 상태를 표시

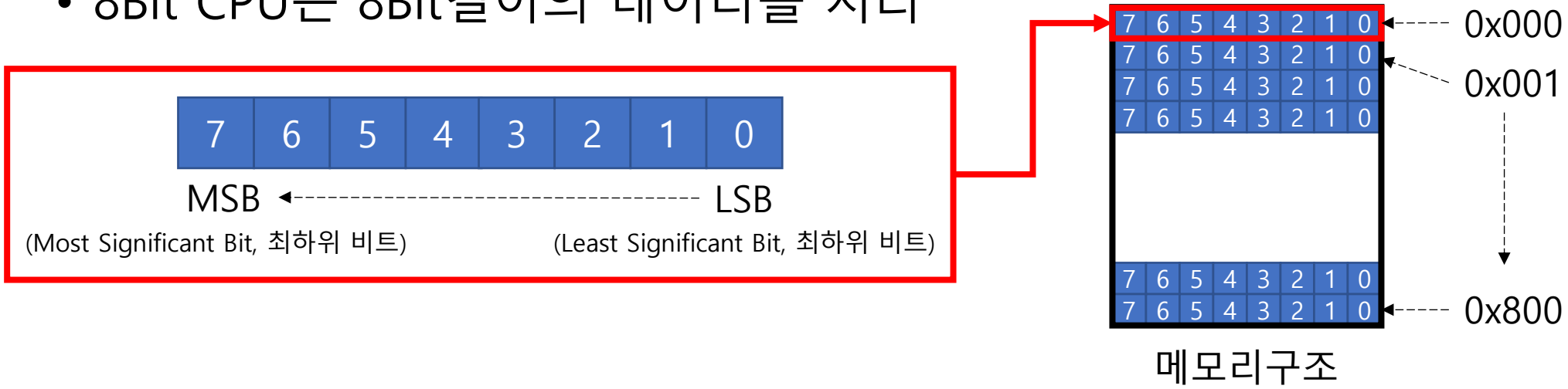
범용 레지스터 (General Purpose Register)

- 프로그램 수행 중에 중간 결과나 데이터를 일시적으로 저장하는데 사용

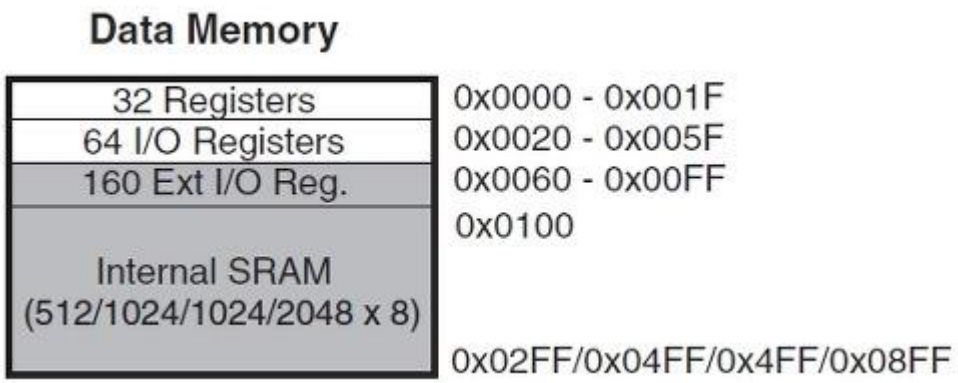
데이타시트 : https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf

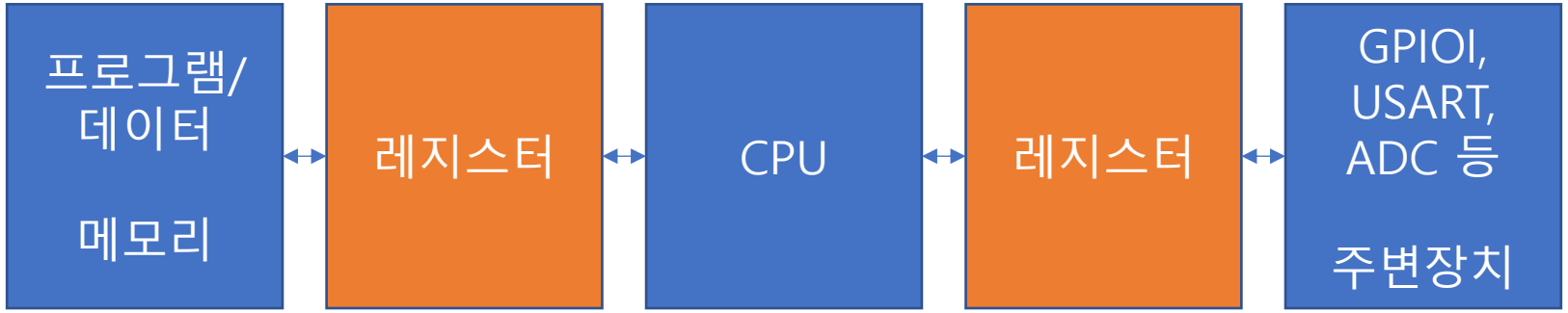
Address란 무엇인가?

- 메모리의 장소 정보 (주소)
  - 동서울대학교 : 경기 성남시 수정구 복정로 76
- 메모리도 주소(연속숫자)를 이용하여 데이터를 참조 한다.
- ATmega328p의 경우 내부에 2KByte의 RAM을 가지고 있다.
  - 1Byte → 8Bit
  - 8Bit CPU는 8Bit길이의 데이터를 처리



ATMEGA328P의 메모리맵과 레지스터

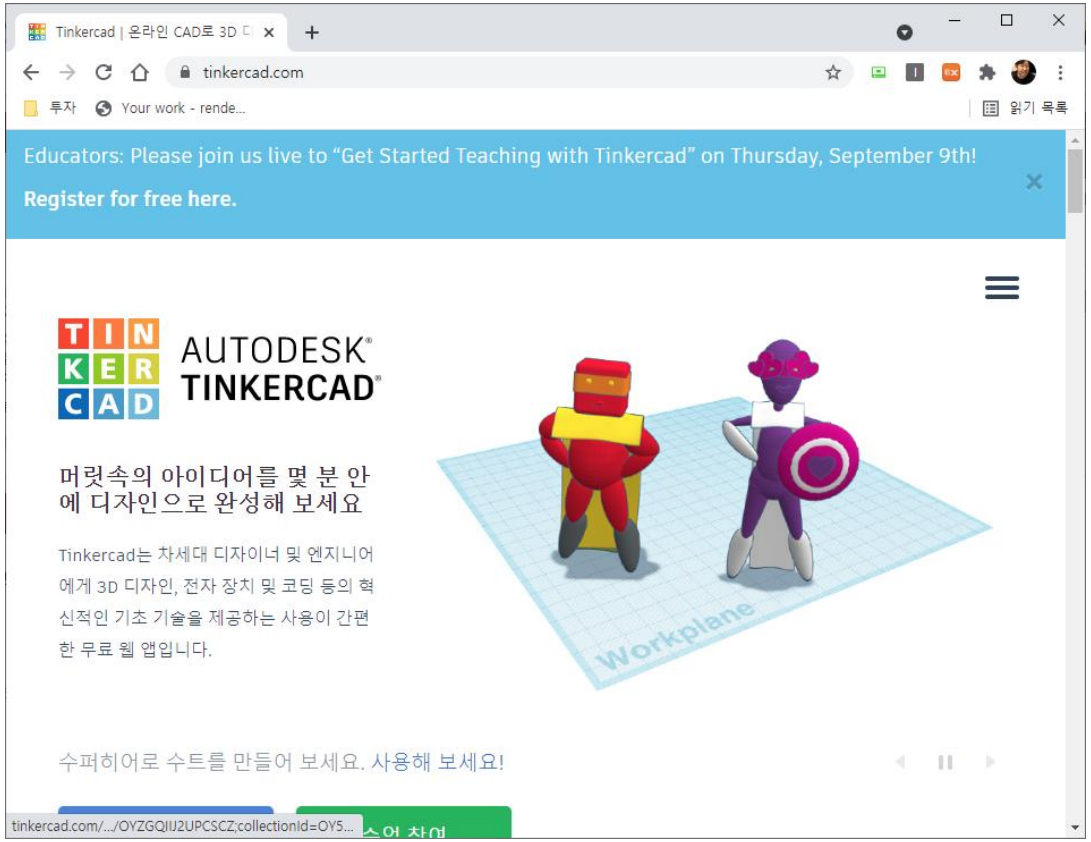


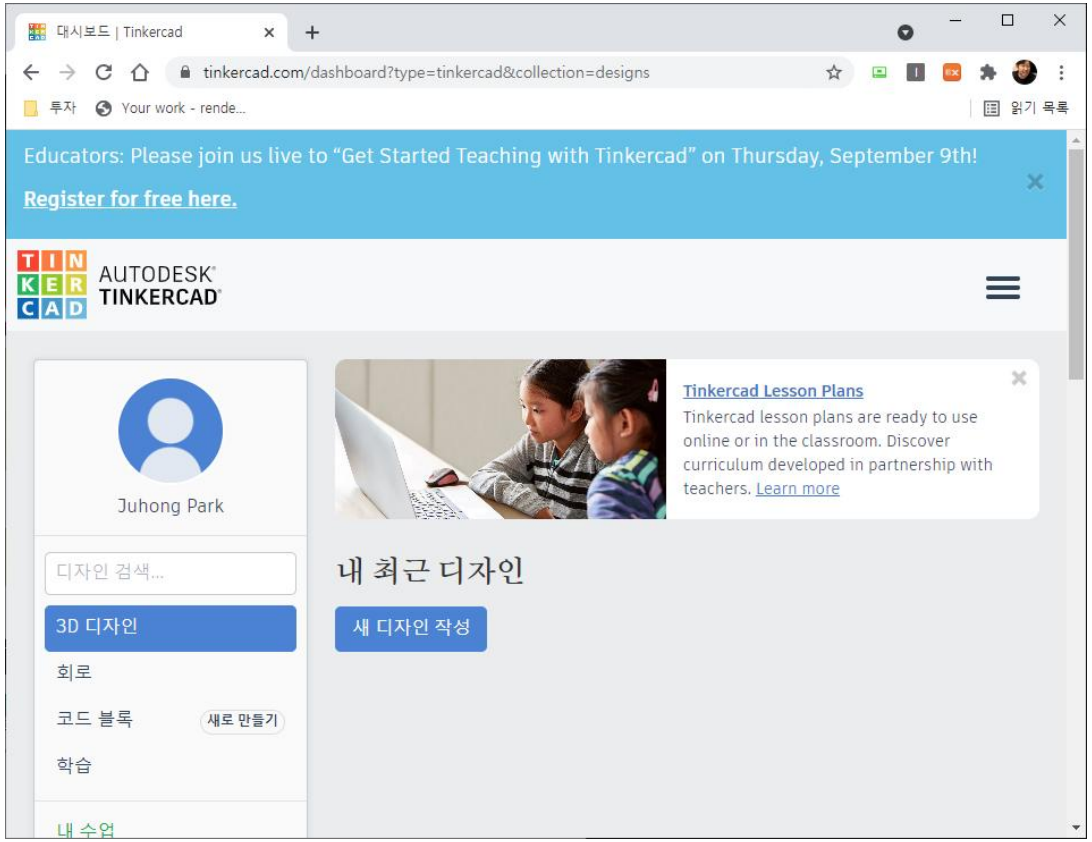


데이타시트 : https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf
30. Register Summary, 275 page 참고

Tinkercad를 활용한 아두이노 실험

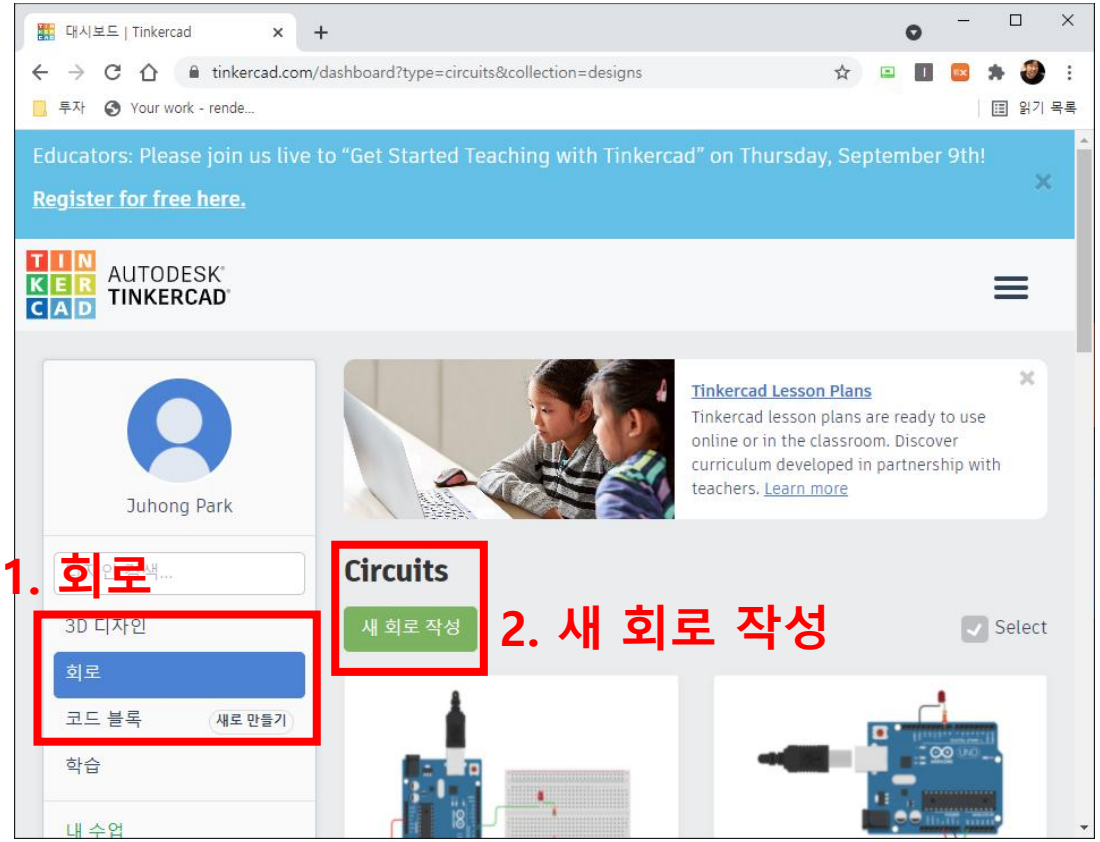
- tinkercad.com

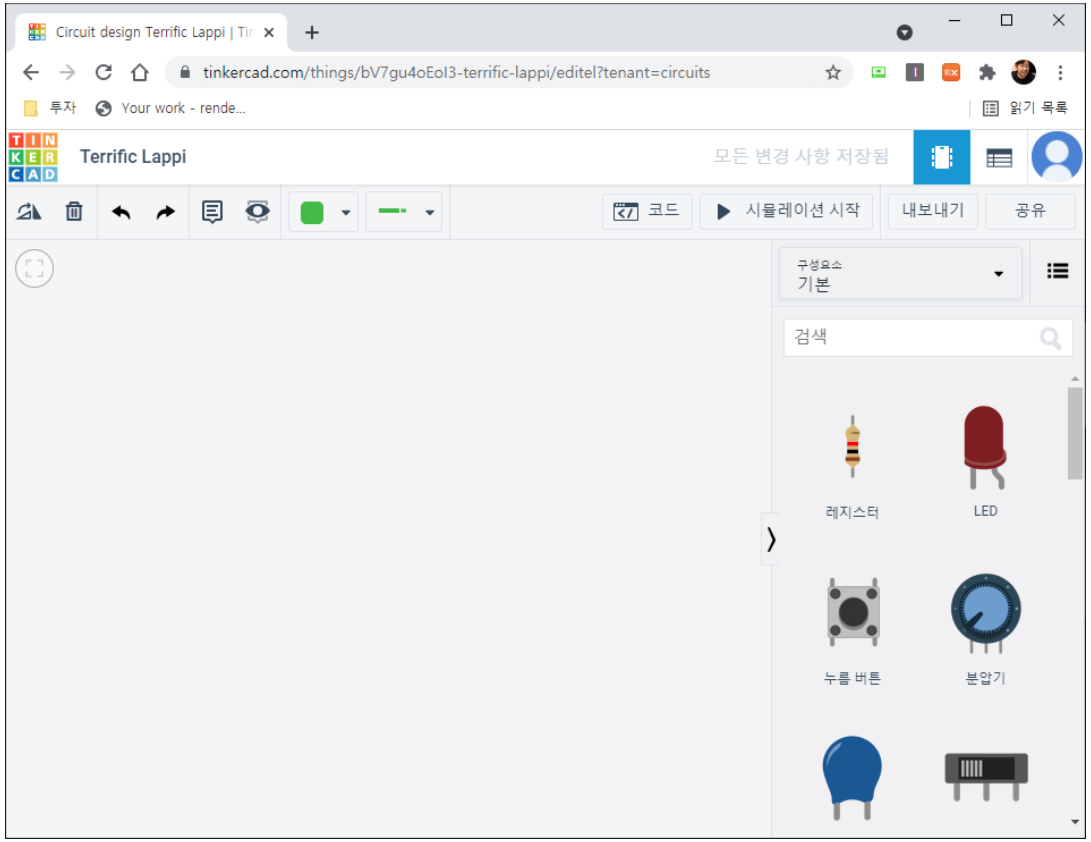




Tinkercad를 활용한 아두이노 실험

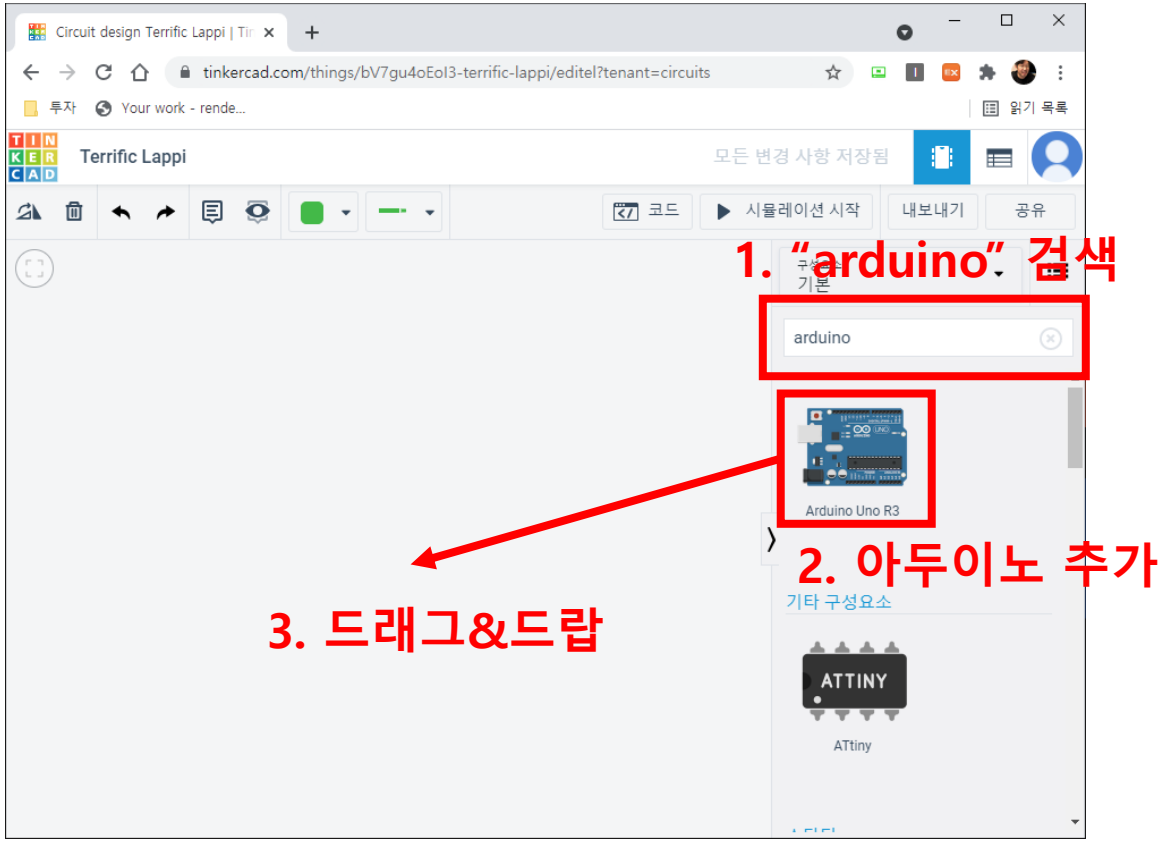
- tinkercad.com : 새 회로 작성

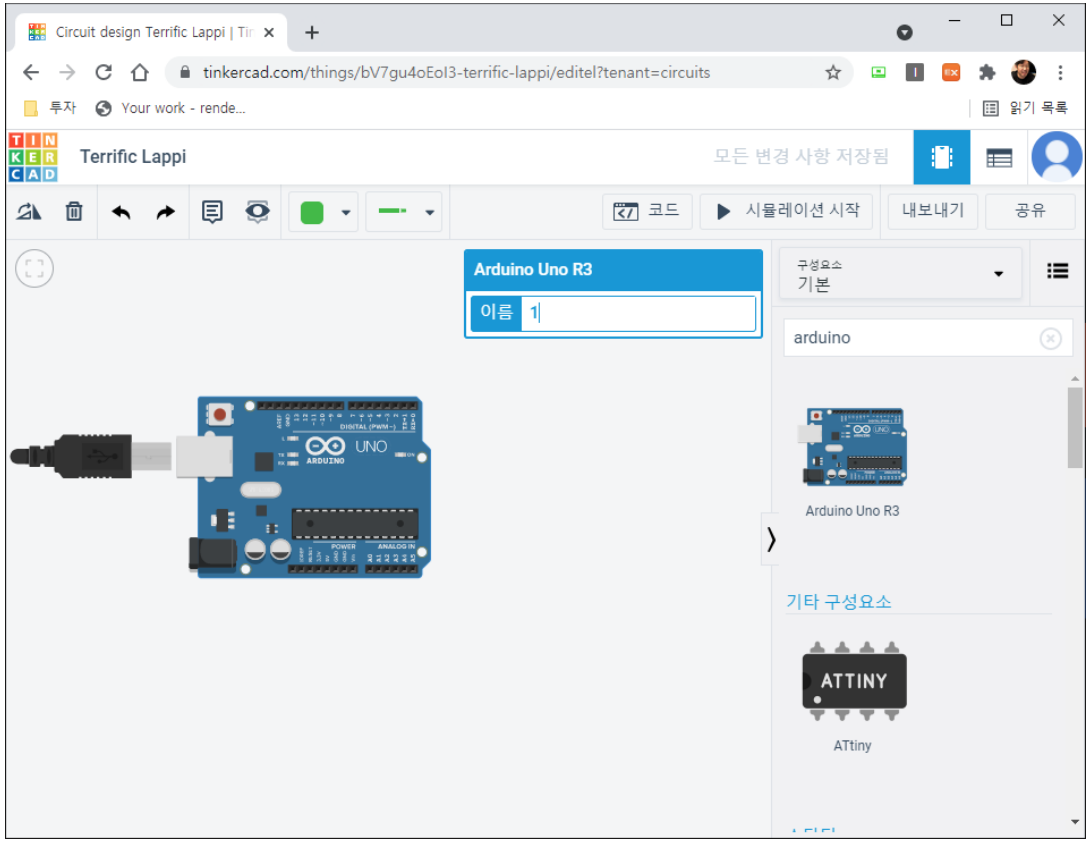




Tinkercad를 활용한 아두이노 실험

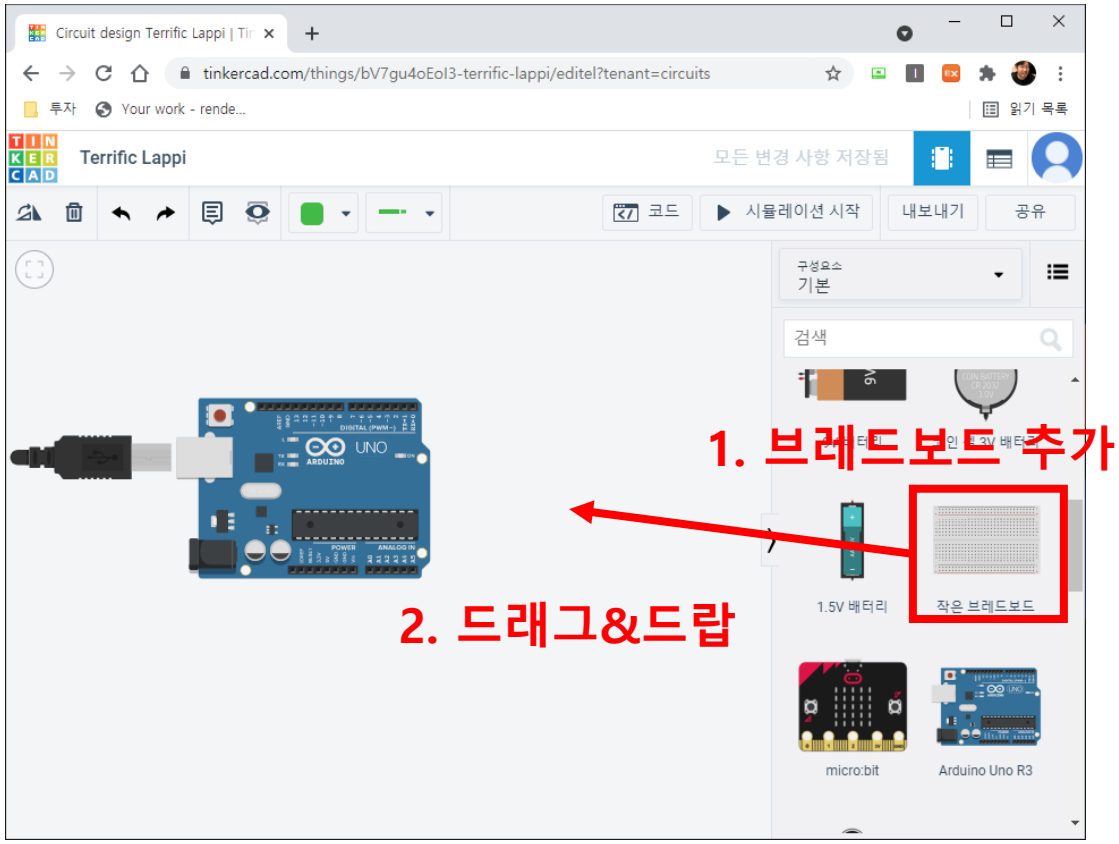
- tinkercad.com : 아두이노 추가

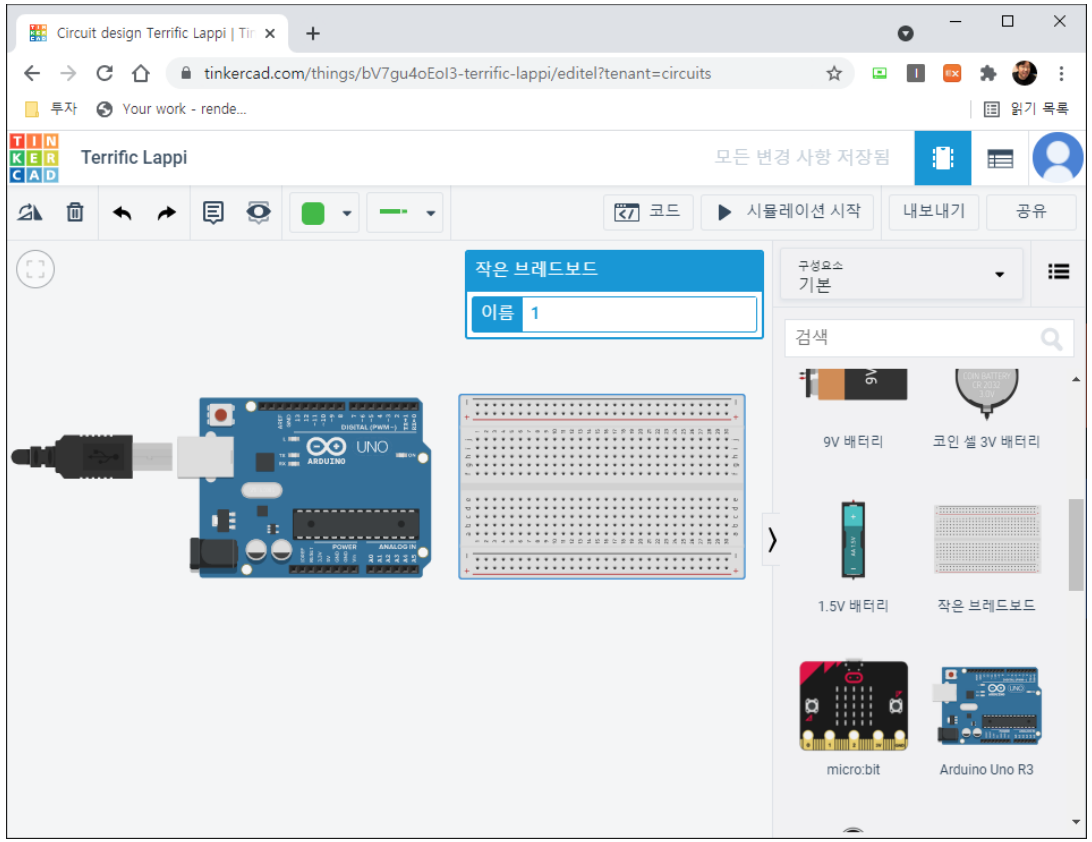




Tinkercad를 활용한 아두이노 실험

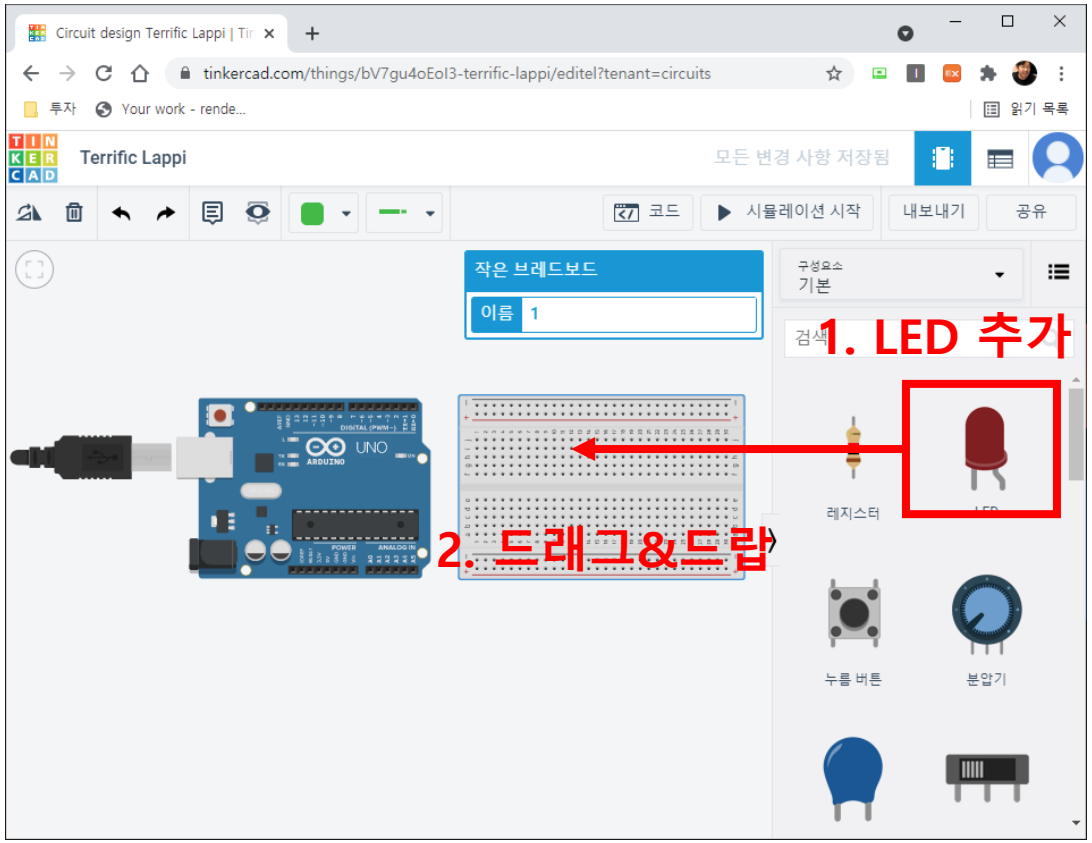
- tinkercad.com : 브레드보드 추가

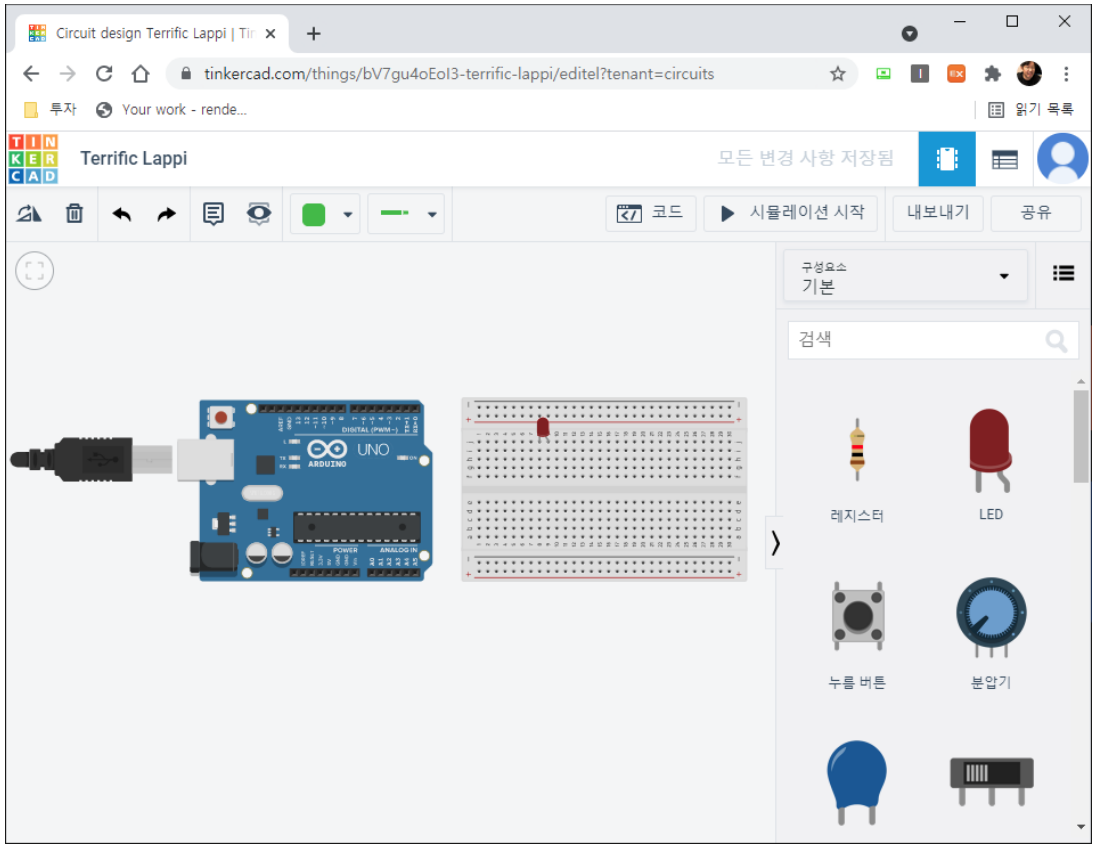




Tinkercad를 활용한 아두이노 실험

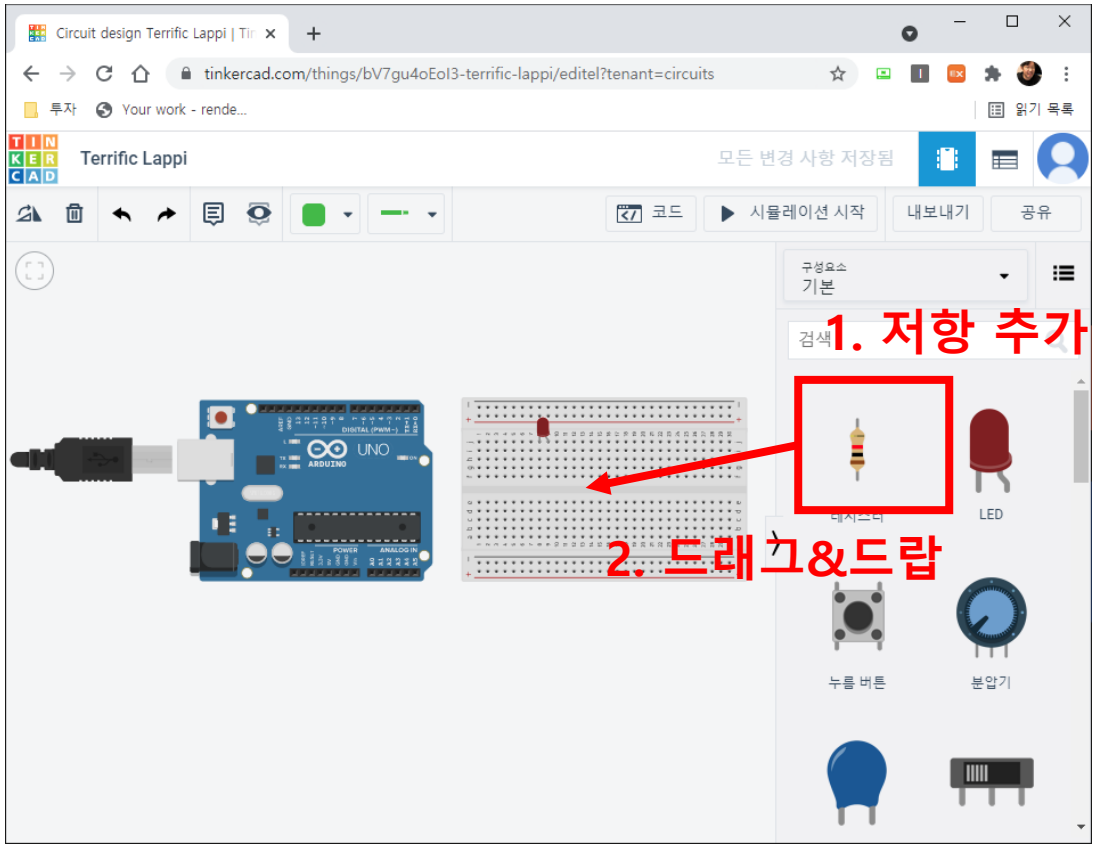
- tinkercad.com : LED 추가

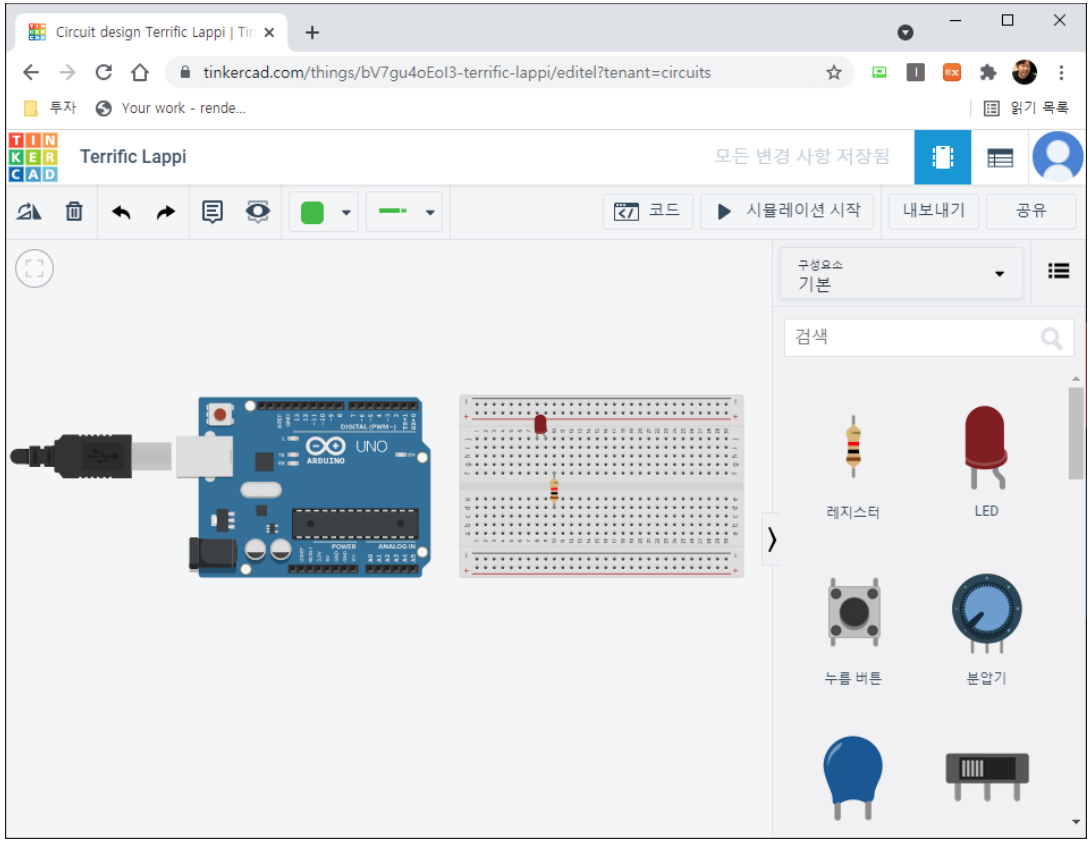




Tinkercad를 활용한 아두이노 실험

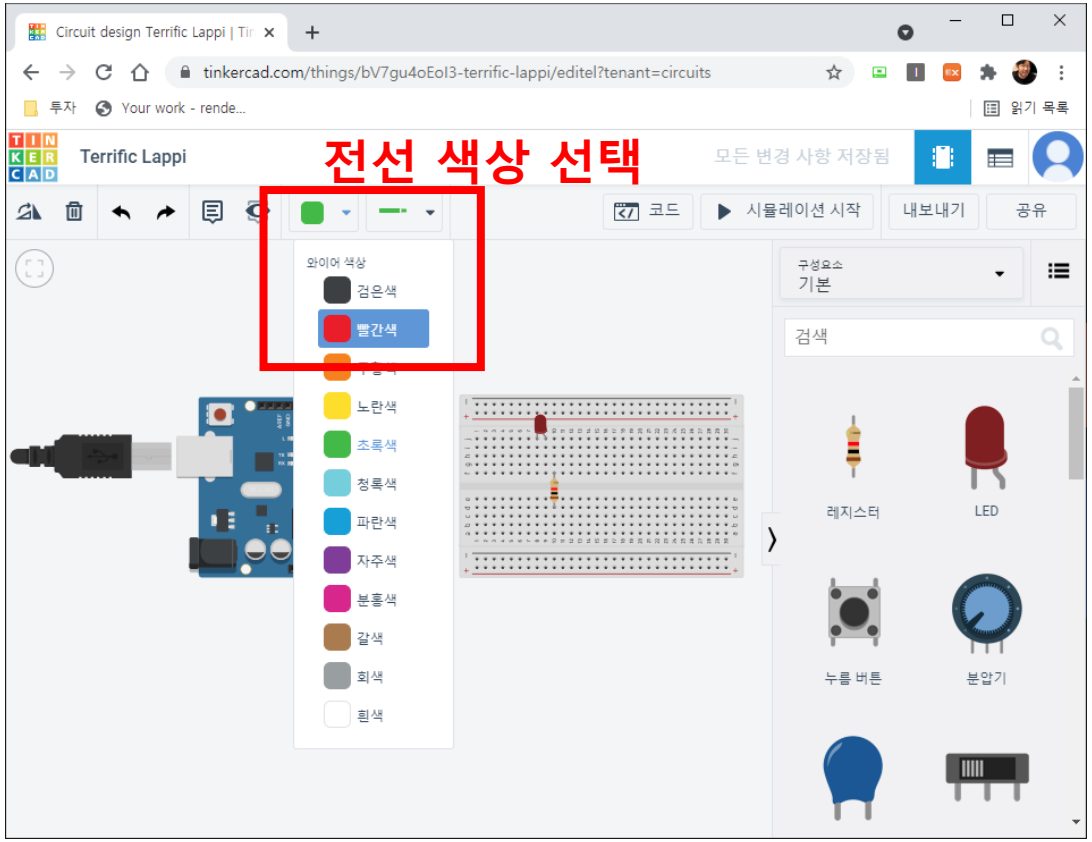
- tinkercad.com : 저항 추가

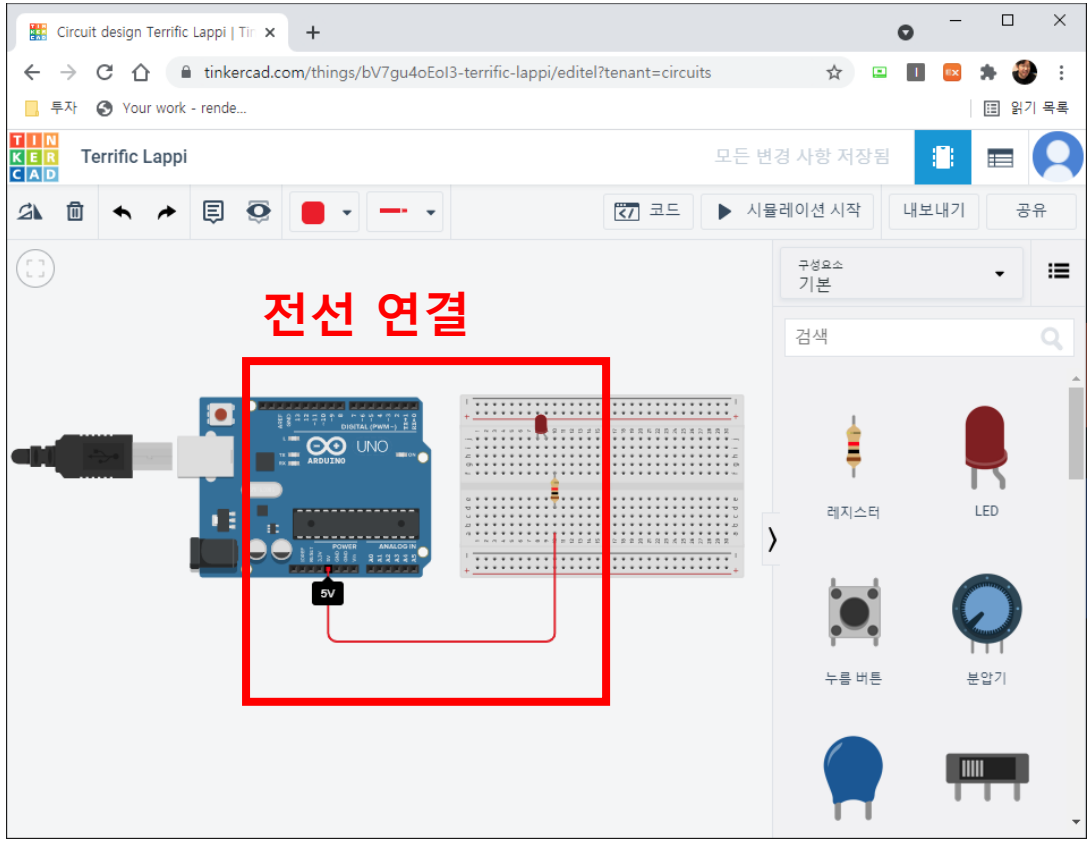




Tinkercad를 활용한 아두이노 실험

- tinkercad.com : 5V(+파워 연결)



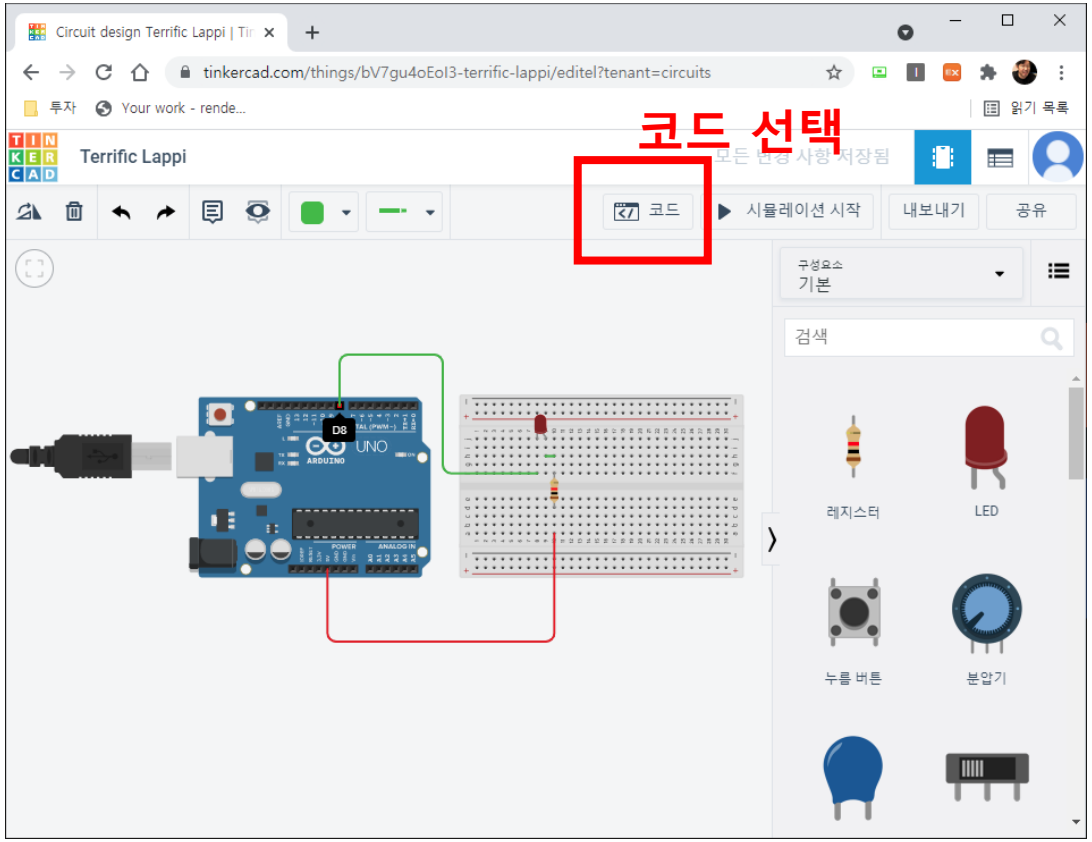


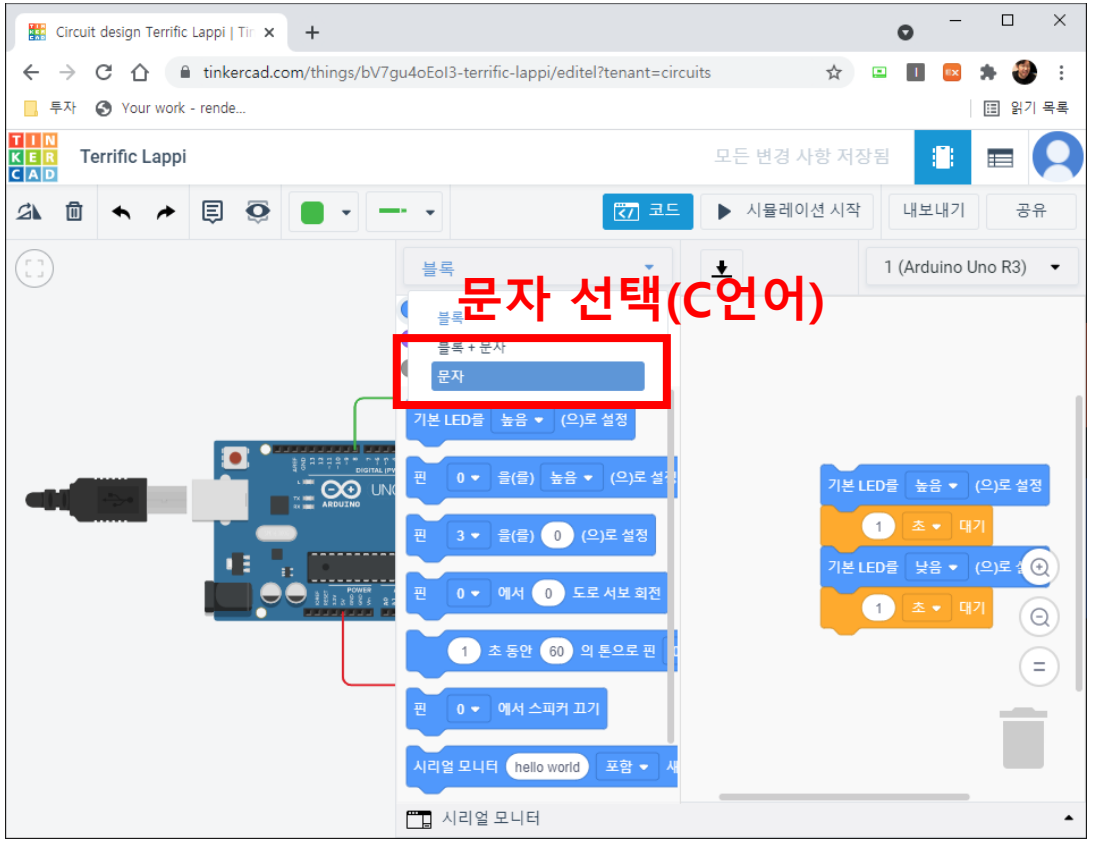
Tinkercad를 활용한 아두이노 실험

- tinkercad.com : 전체 회로 구성

Tinkercad를 활용한 아두이노 실험

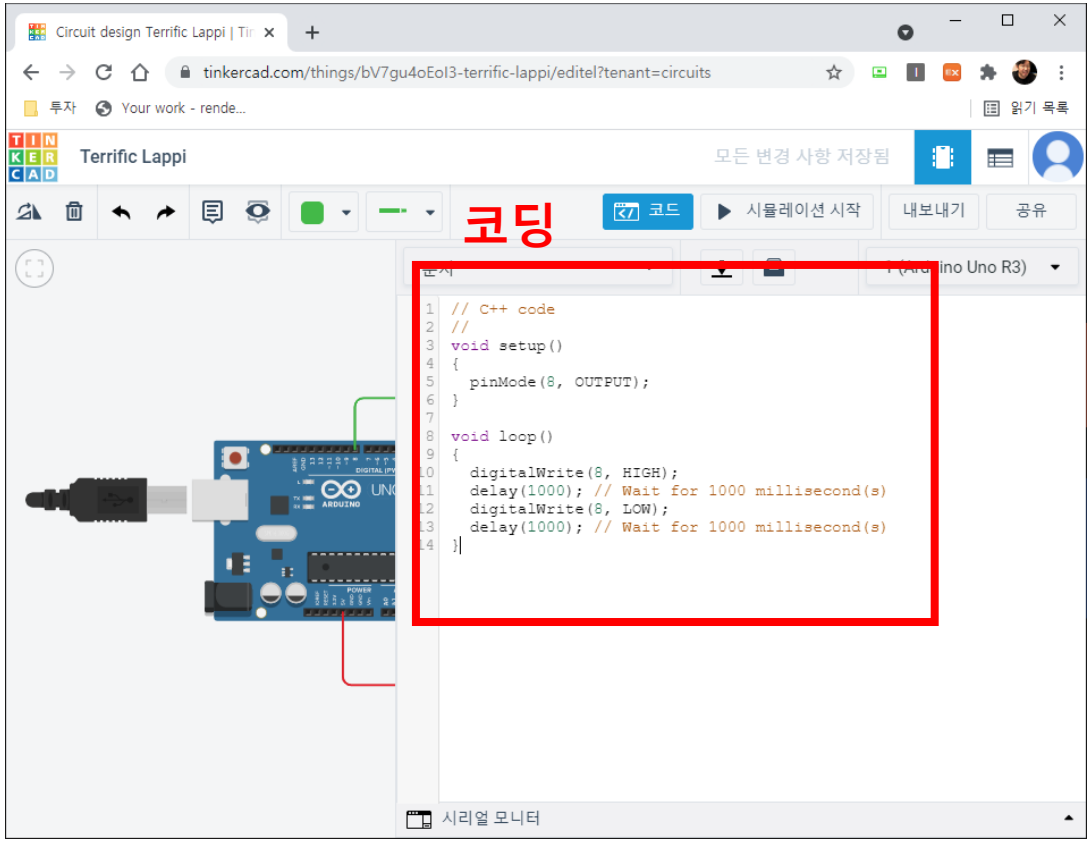
- tinkercad.com : 코드 작성





Tinkercad를 활용한 아두이노 실험

- tinkercad.com : 코드 작성

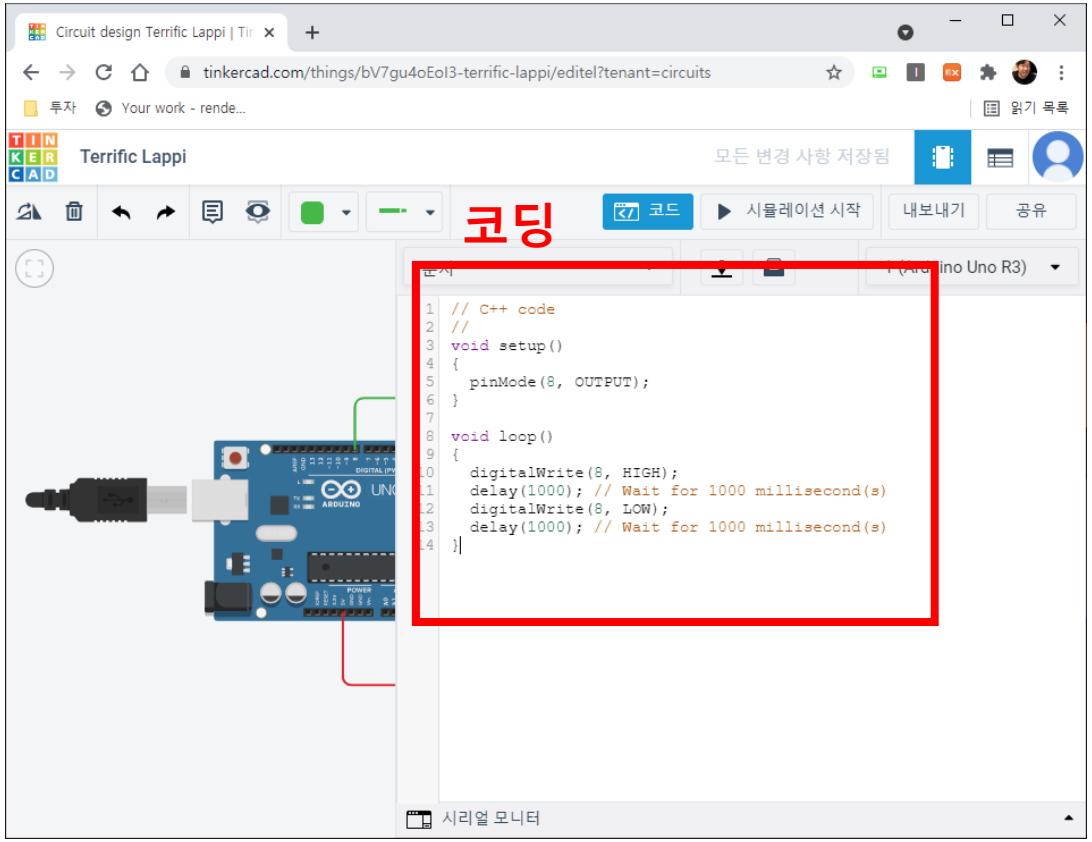



```
// C++ code
//
void setup()
{
  pinMode(8, OUTPUT);
}

void loop()
{
  digitalWrite(8, HIGH);
  delay(1000); // Wait for 1000 millisecond(s)
  digitalWrite(8, LOW);
  delay(1000); // Wait for 1000 millisecond(s)
}
```

Tinkercad를 활용한 아두이노 실험

- tinkercad.com : 코드 작성 with 챗피티

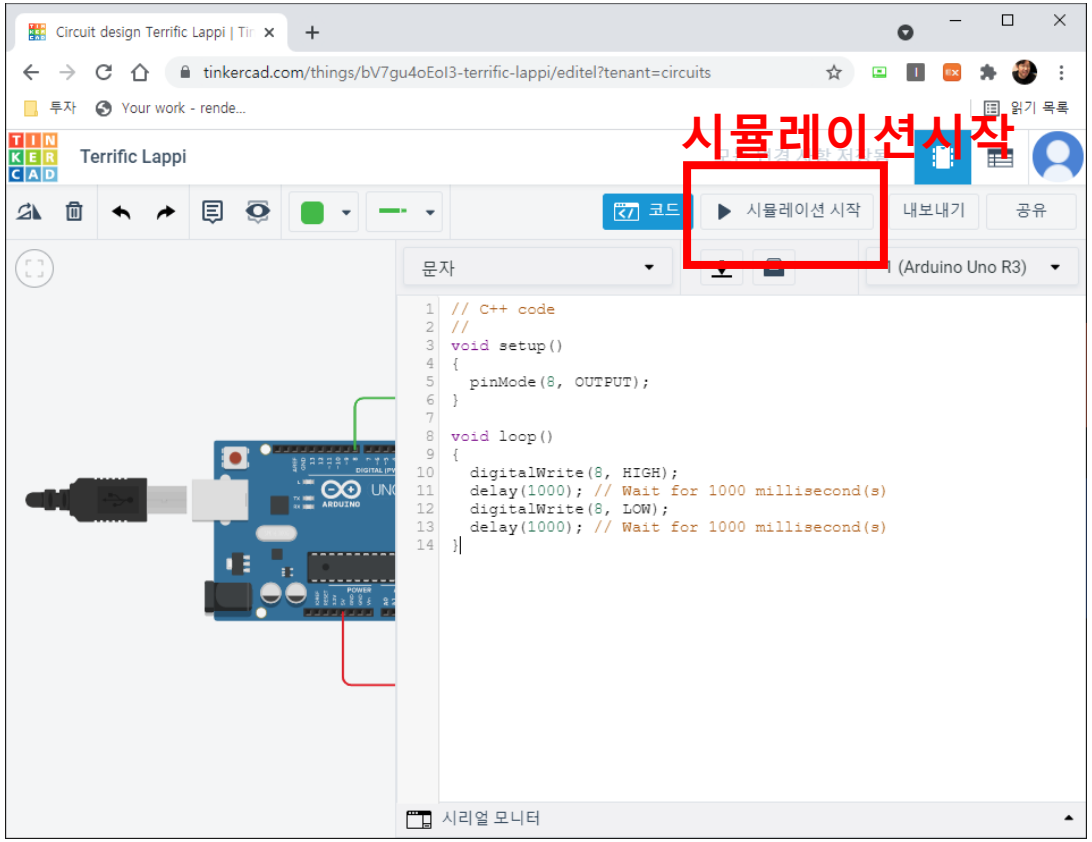
아두이노 우노의 8번핀에 LED -핀을 연결했어. LED를 껐다 켜는 것을 반복하는 아두이노 코드 작성해줘

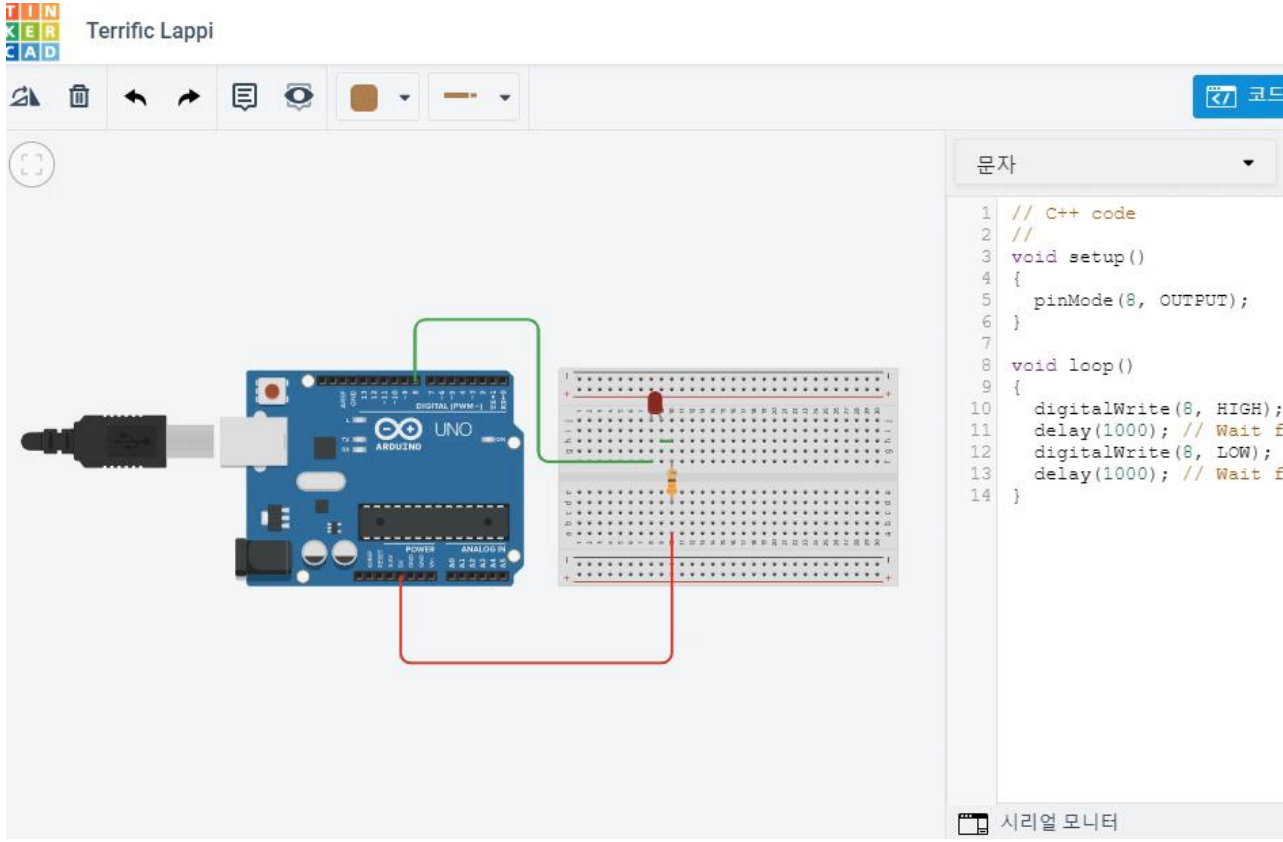



```
// C++ code
//
void setup()
{
  pinMode(8, OUTPUT);
}

void loop()
{
  digitalWrite(8, HIGH);
  delay(1000); // Wait for 1000 millisecond(s)
  digitalWrite(8, LOW);
  delay(1000); // Wait for 1000 millisecond(s)
}
```

Tinkercad를 활용한 아두이노 실험

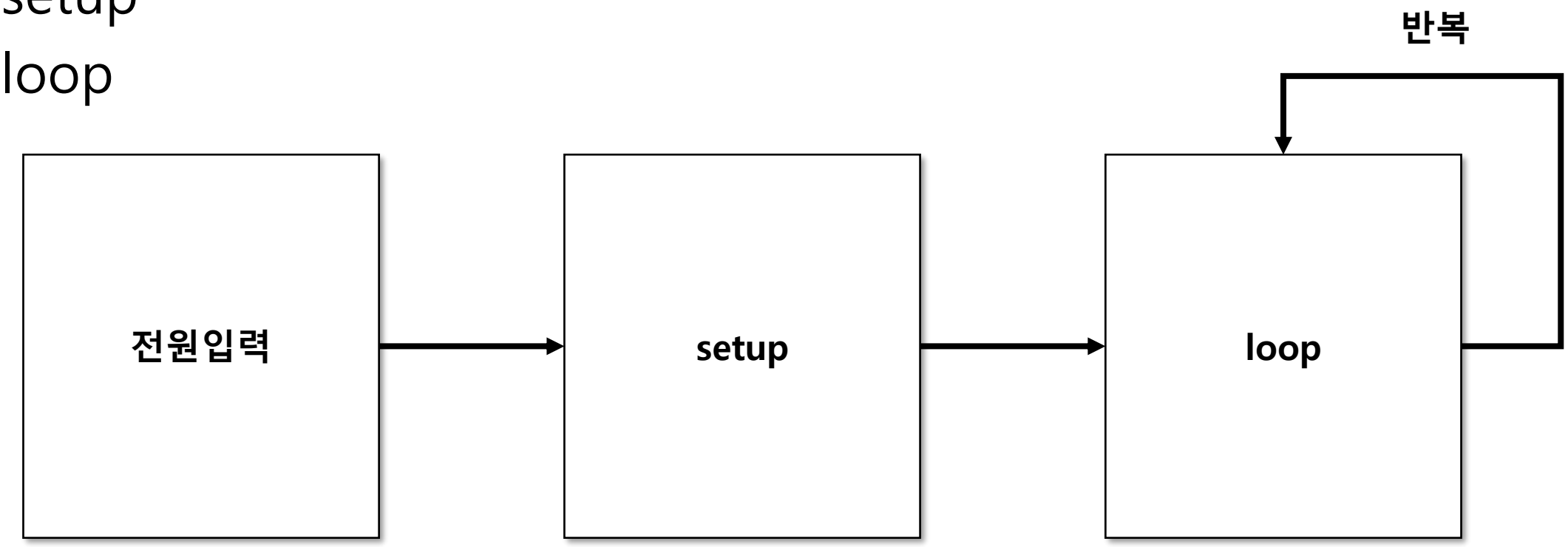
- tinkercad.com : 시뮬레이션 시작



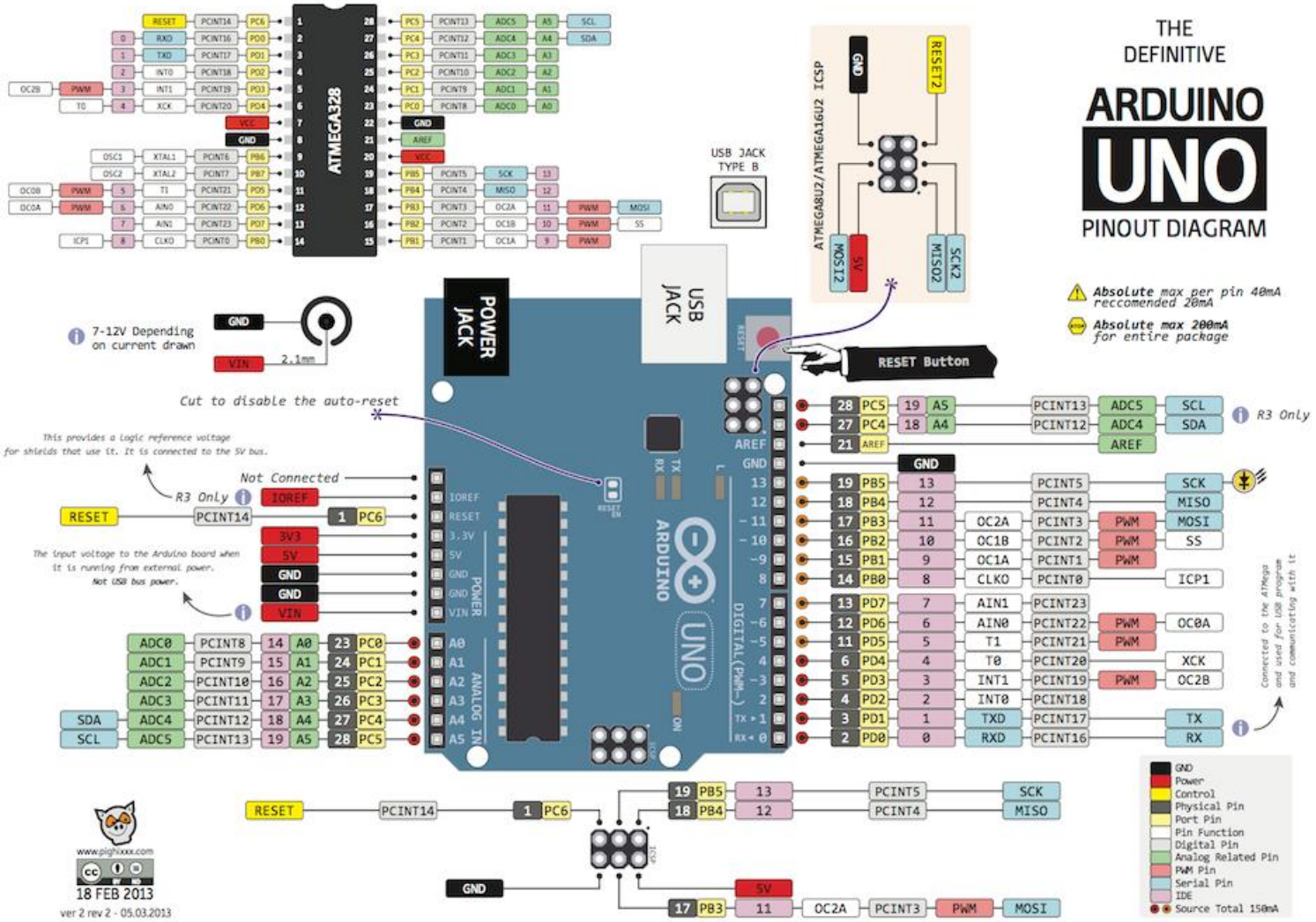


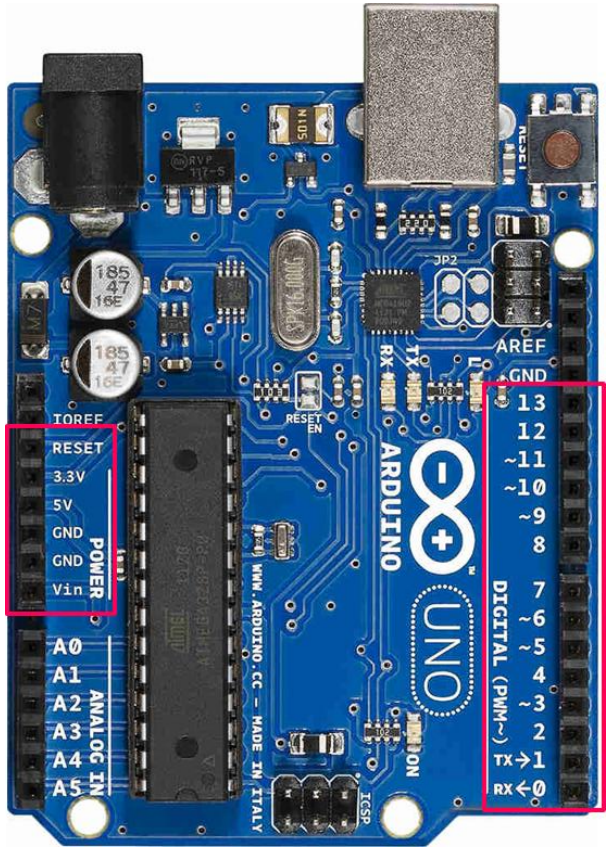
Tinkercad를 활용한 아두이노 실험

- setup
- loop

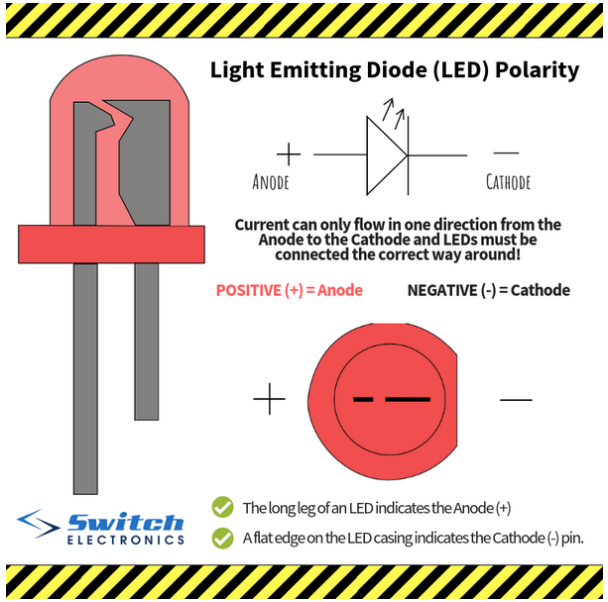


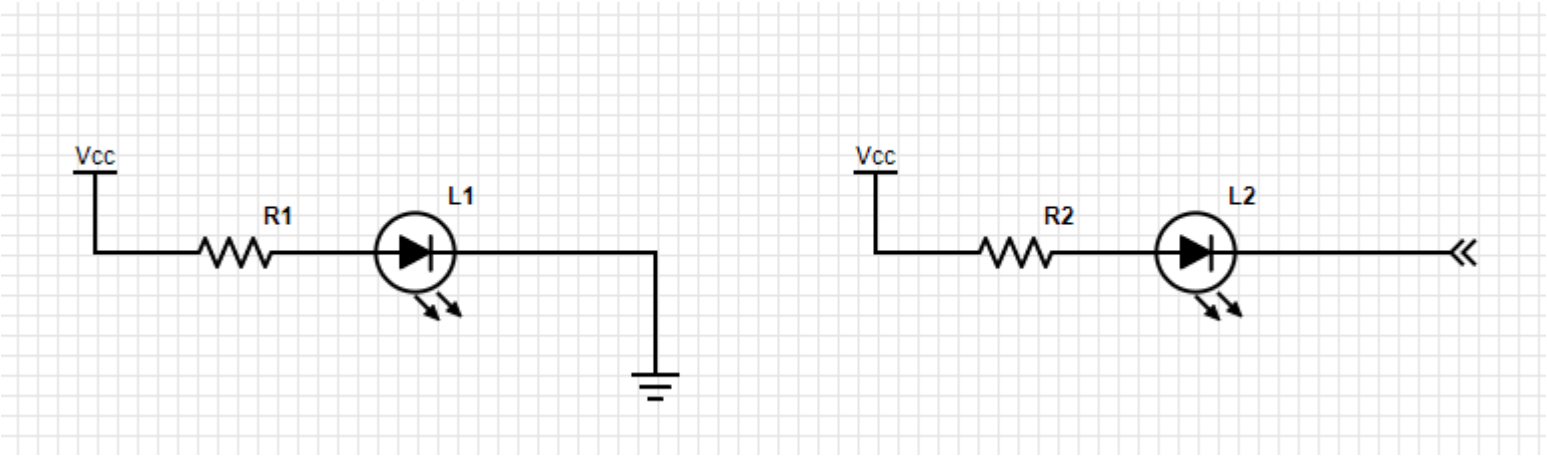
아두이노 디지털 IO


THE DEFINITIVE
ARDUINO
UNO
PINOUT DIAGRAM
ATMEGA328
ATMEGA8U2/ATMEGA16U2 ICSP
USB JACK TYPE B
POWER JACK
USB JACK
RESET Button
7-12V Depending on current drawn
Cut to disable the auto-reset
Not Connected
R3 Only
Absolute max per pin 40mA reccomended 20mA
Absolute max 200mA for entire package
18 FEB 2013
ver 2 rev 2 - 05.03.2013


IOREF
RESET
3.3V
5V
GND
GND
Vin
POWER
A0
A1
A2
A3
A4
A5
ANALOG IN
DIGITAL (PWM~)
AREF
GND
13
12
~11
~10
~9
8
7
~6
~5
4
~3
2
TX→1
RX←0
ARDUINO
UNO
WWW.ARDUINO.CC – MADE IN ITALY

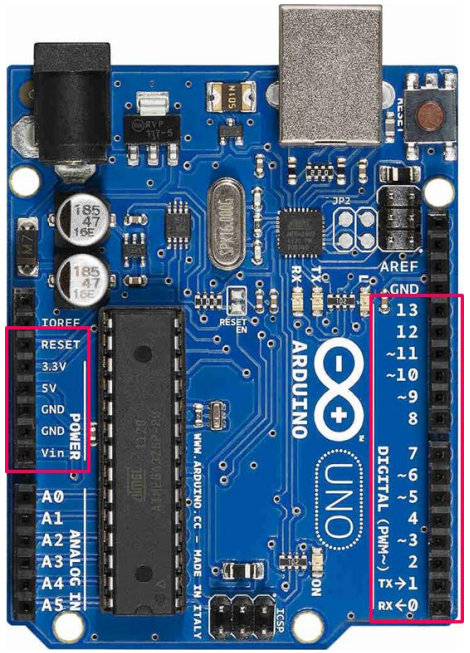
아두이노 디지털 IO

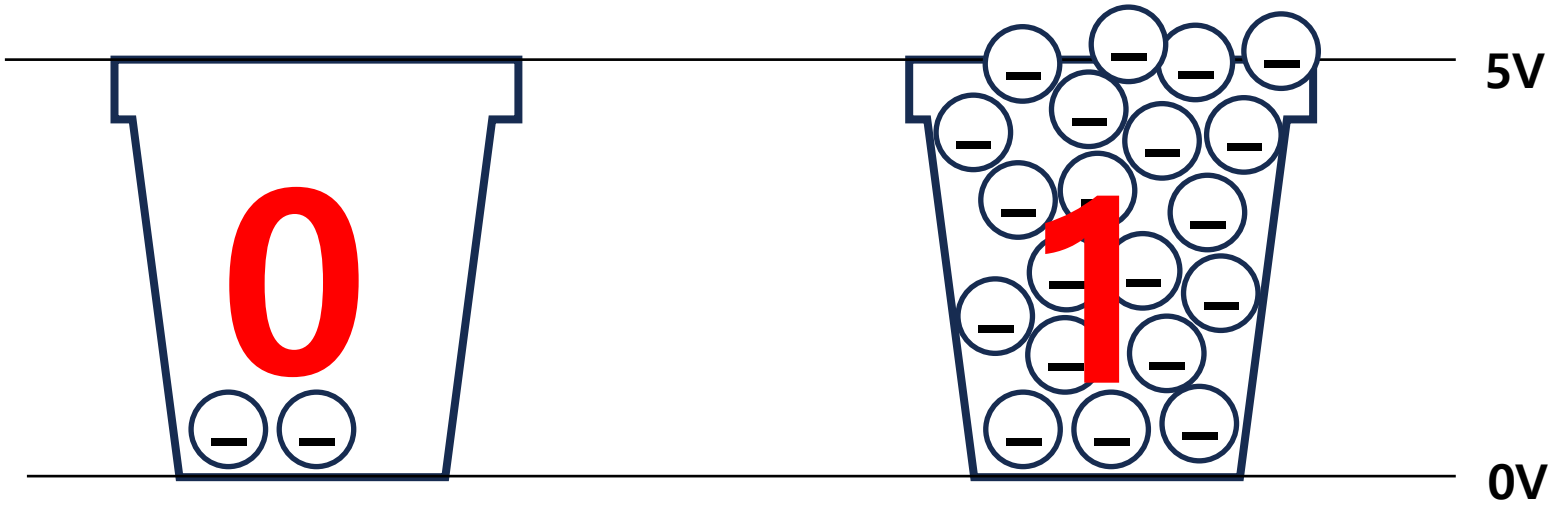

Light Emitting Diode (LED) Polarity
+
ANODE
—
CATHODE
Current can only flow in one direction from the Anode to the Cathode and LEDs must be connected the correct way around!
POSITIVE (+) = Anode
NEGATIVE (-) = Cathode
+
—
Switch ELECTRONICS
The long leg of an LED indicates the Anode (+)
A flat edge on the LED casing indicates the Cathode (-) pin.


Vcc
R1
L1
Vcc
R2
L2

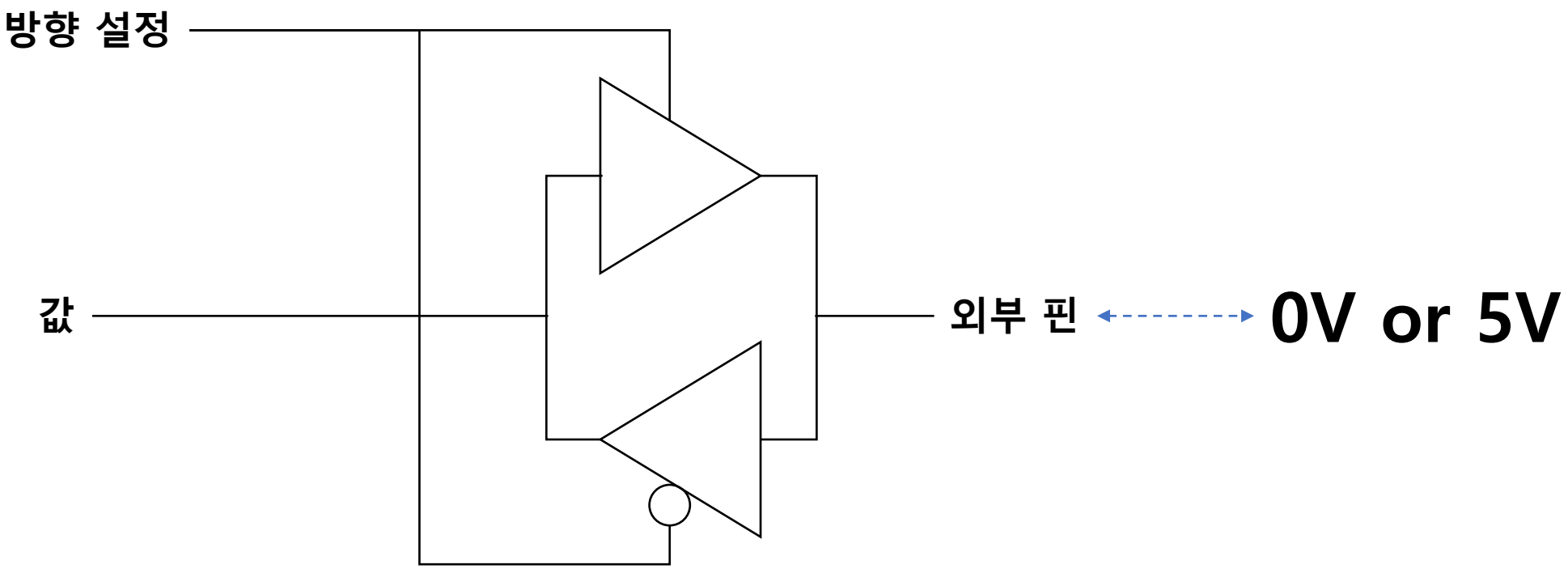
디지털 IO(Input/Output)

- 전자 장치 및 시스템에서 **신호를 주고받는 기본적인 방식**
- 신호는 **0(LOW, 0V)** 또는 **1(HIGH, 5V)**로 표현
- 다양한 센서, 액추에이터, 마이크로컨트롤러, PLC 등에서 사용





디지털 IO(Input/Output)



디지털 IO 포트 관련 레지스터

- Port

13.4.2 PORTB – The Port B Data Register

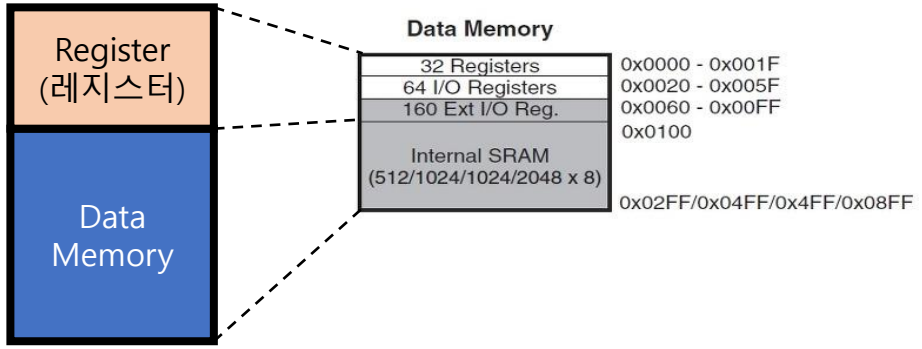
Bit	7	6	5	4	3	2	1	0	
0x05 (0x25)	PORTB7	PORTB6	PORTB5	PORTB4	PORTB3	PORTB2	PORTB1	PORTB0	PORTB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

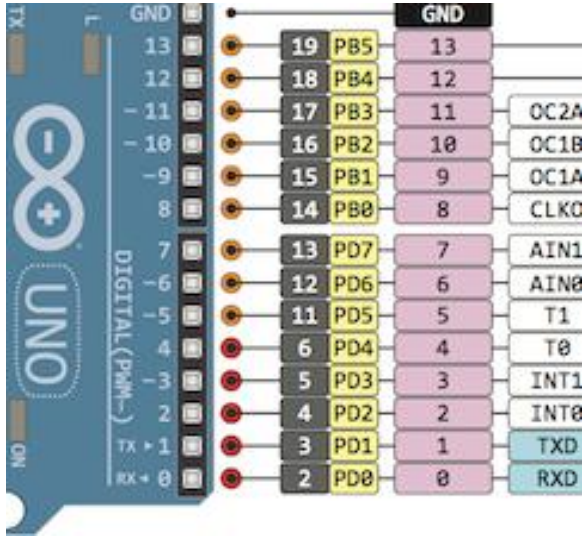
13.4.3 DDRB – The Port B Data Direction Register

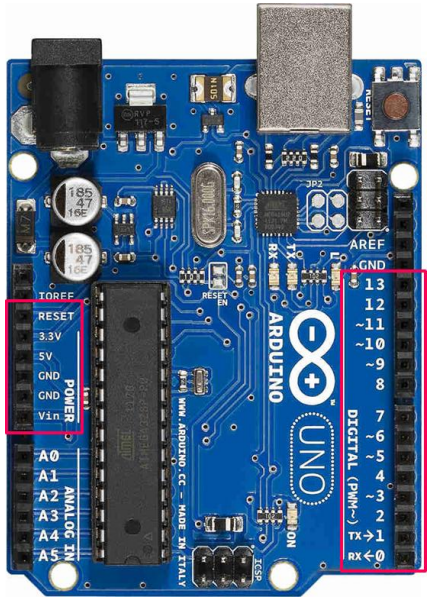
Bit	7	6	5	4	3	2	1	0	
0x04 (0x24)	DDB7	DDB6	DDB5	DDB4	DDB3	DDB2	DDB1	DDB0	DDRB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

13.4.4 PINB – The Port B Input Pins Address

Bit	7	6	5	4	3	2	1	0	
0x03 (0x23)	PINB7	PINB6	PINB5	PINB4	PINB3	PINB2	PINB1	PINB0	PINB
Read/Write	R	R	R	R	R	R	R	R	
Initial Value	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	

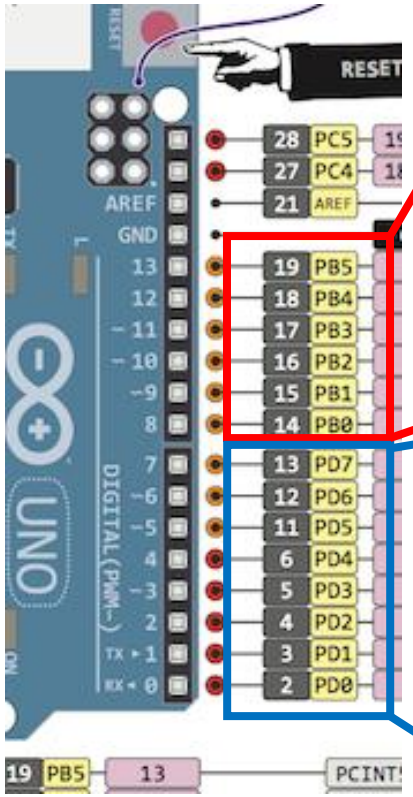






디지털 IO 포트 관련 레지스터

- Port



13.4.2 PORTB – The Port B Data Register

Bit	7	6	5	4	3	2	1	0	
0x05 (0x25)	PORTB7	PORTB6	PORTB5	PORTB4	PORTB3	PORTB2	PORTB1	PORTB0	PORTB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

13.4.3 DDRB – The Port B Data Direction Register

Bit	7	6	5	4	3	2	1	0	
0x04 (0x24)	DDB7	DDB6	DDB5	DDB4	DDB3	DDB2	DDB1	DDB0	DDRB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

13.4.4 PINB – The Port B Input Pins Address

Bit	7	6	5	4	3	2	1	0	
0x03 (0x23)	PINB7	PINB6	PINB5	PINB4	PINB3	PINB2	PINB1	PINB0	PINB
Read/Write	R	R	R	R	R	R	R	R	
Initial Value	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	

13.4.8 PORTD – The Port D Data Register

Bit	7	6	5	4	3	2	1	0	
0x0B (0x2B)	PORTD7	PORTD6	PORTD5	PORTD4	PORTD3	PORTD2	PORTD1	PORTD0	PORTD
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

13.4.9 DDRD – The Port D Data Direction Register

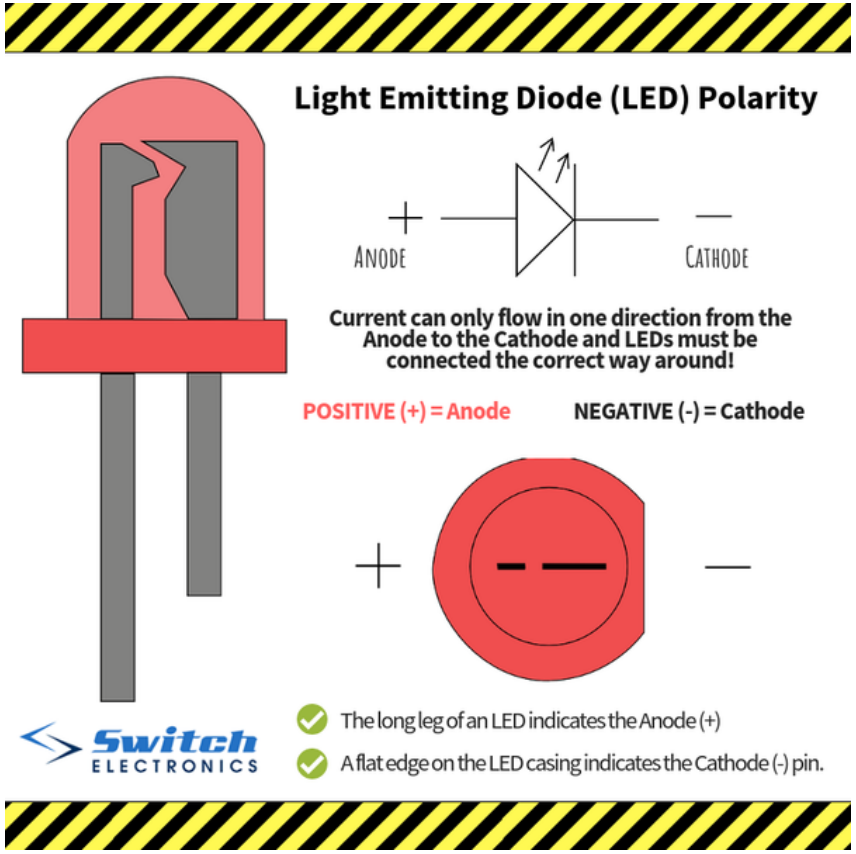
Bit	7	6	5	4	3	2	1	0	
0x0A (0x2A)	DDD7	DDD6	DDD5	DDD4	DDD3	DDD2	DDD1	DDD0	DDRD
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

13.4.10 PIND – The Port D Input Pins Address

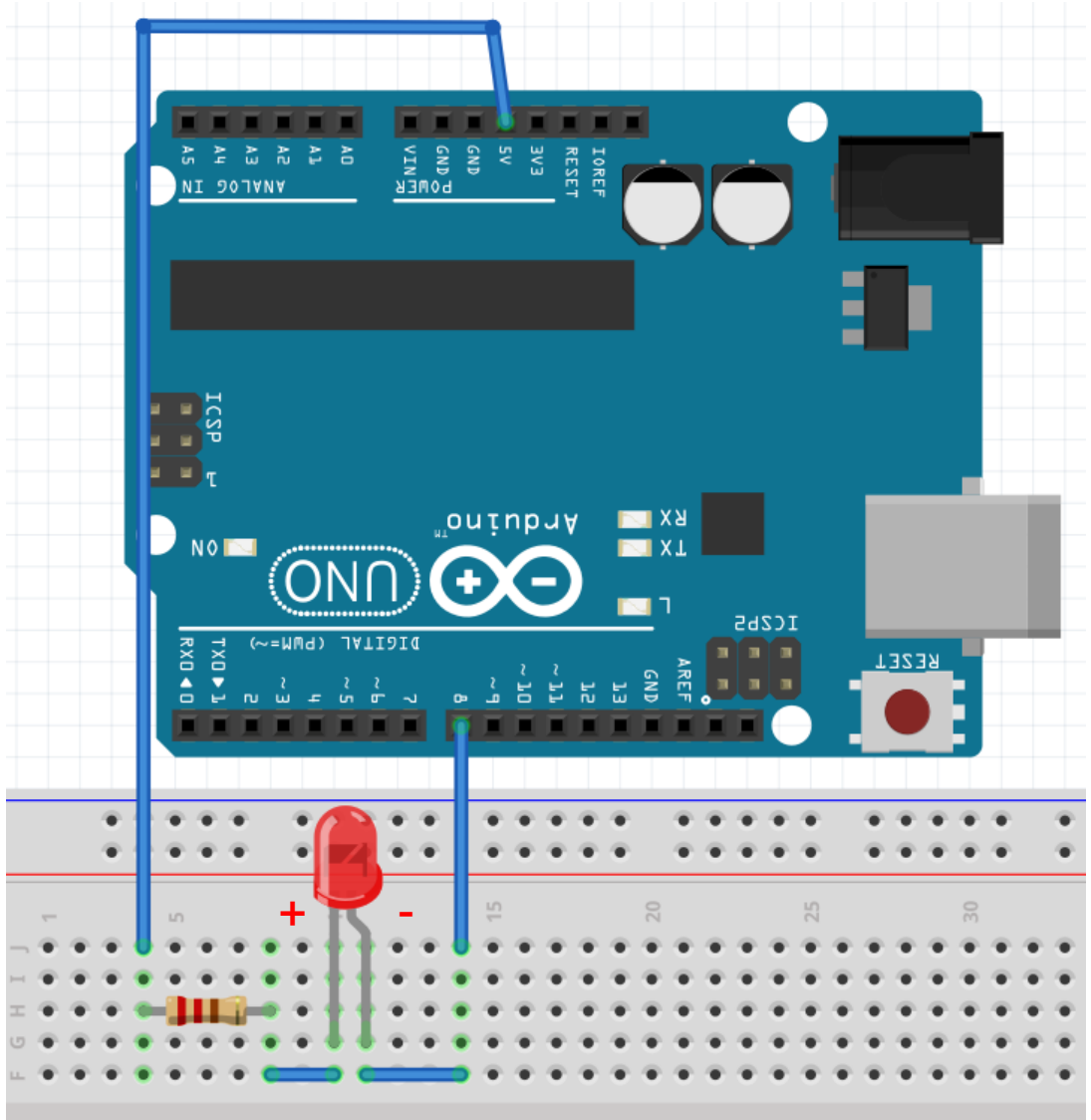
Bit	7	6	5	4	3	2	1	0	
0x09 (0x29)	PIND7	PIND6	PIND5	PIND4	PIND3	PIND2	PIND1	PIND0	PIND
Read/Write	R	R	R	R	R	R	R	R	
Initial Value	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	

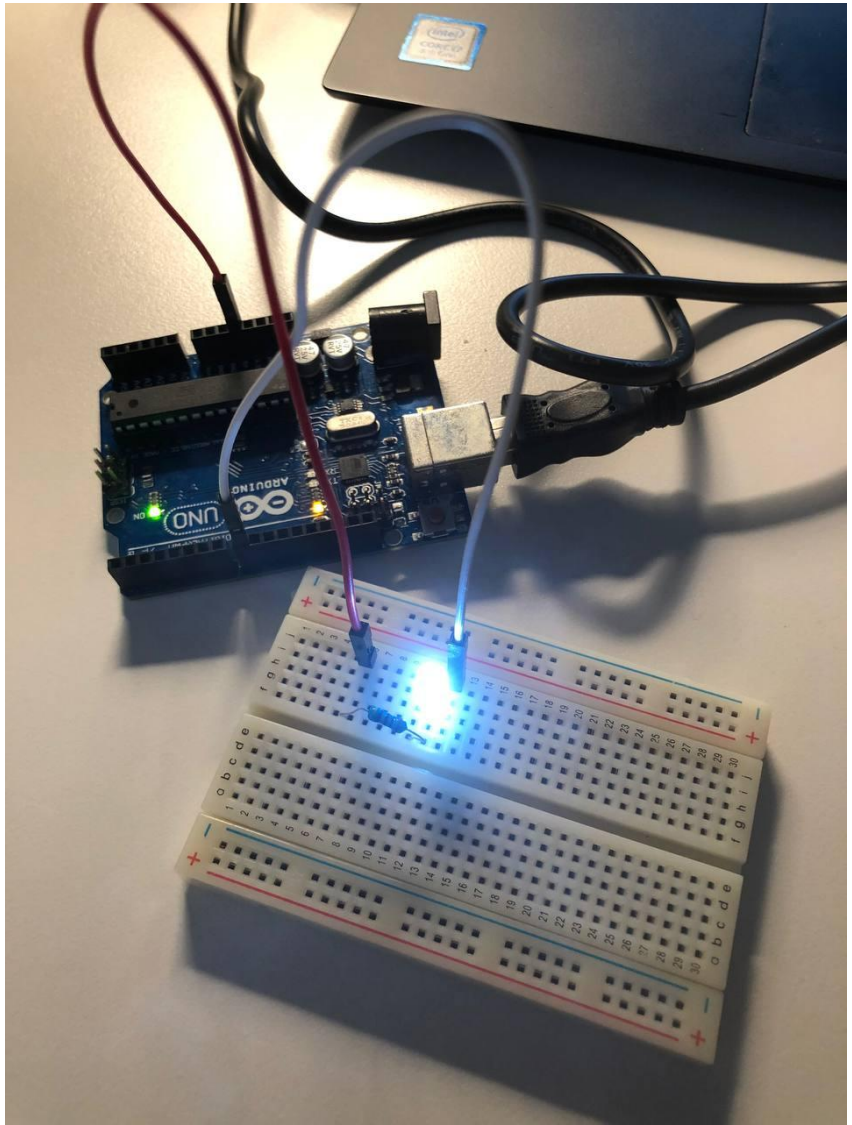
IO 포트 출력 실험

- LED를 이용한 포트 Output 테스트



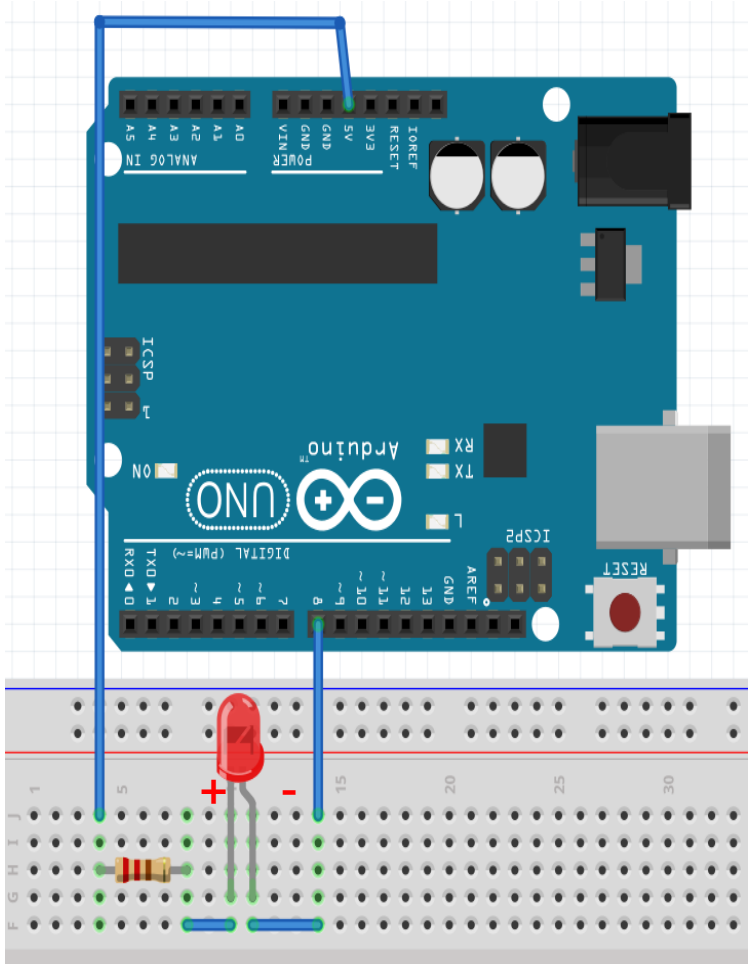
IO 포트 출력 실험

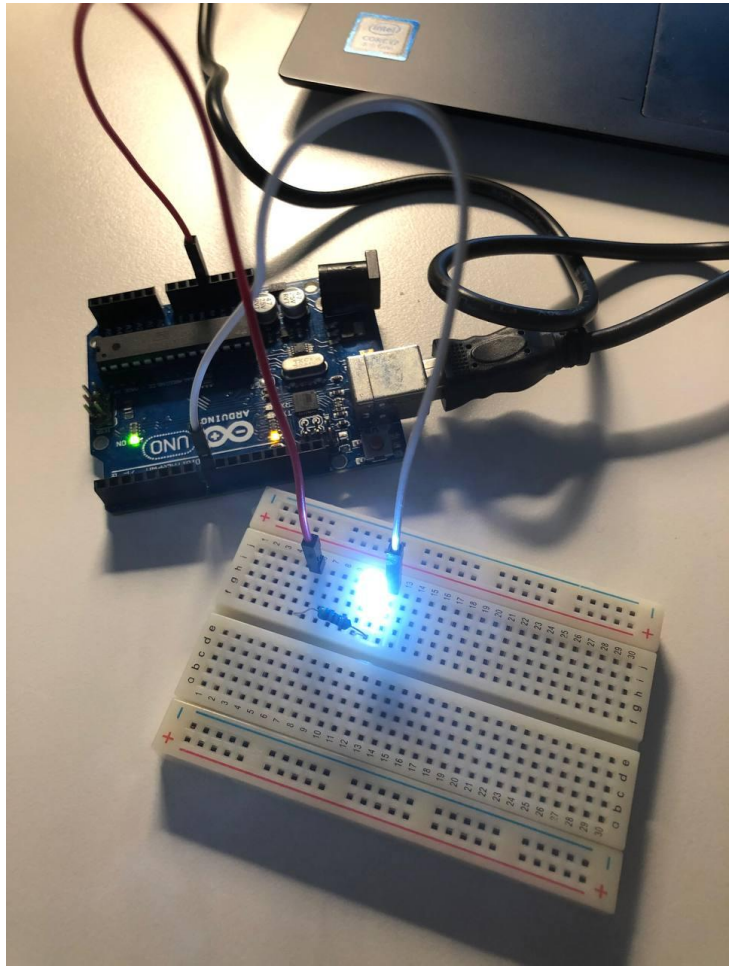

ANALOG IN
A5 A4 A3 A2 A1 A0
POWER
VIN GND GND 5V 3V3 RESET IOREF
ICSP
ON
Arduino
UNO
RX
TX
L
ICSP2
RESET
DIGITAL (PWM=~)
RX0 TX1 2 ~3 4 ~5 ~6 7 8 ~9 ~10 ~11 12 13 GND AREF
+
-



IO 포트 출력 실험

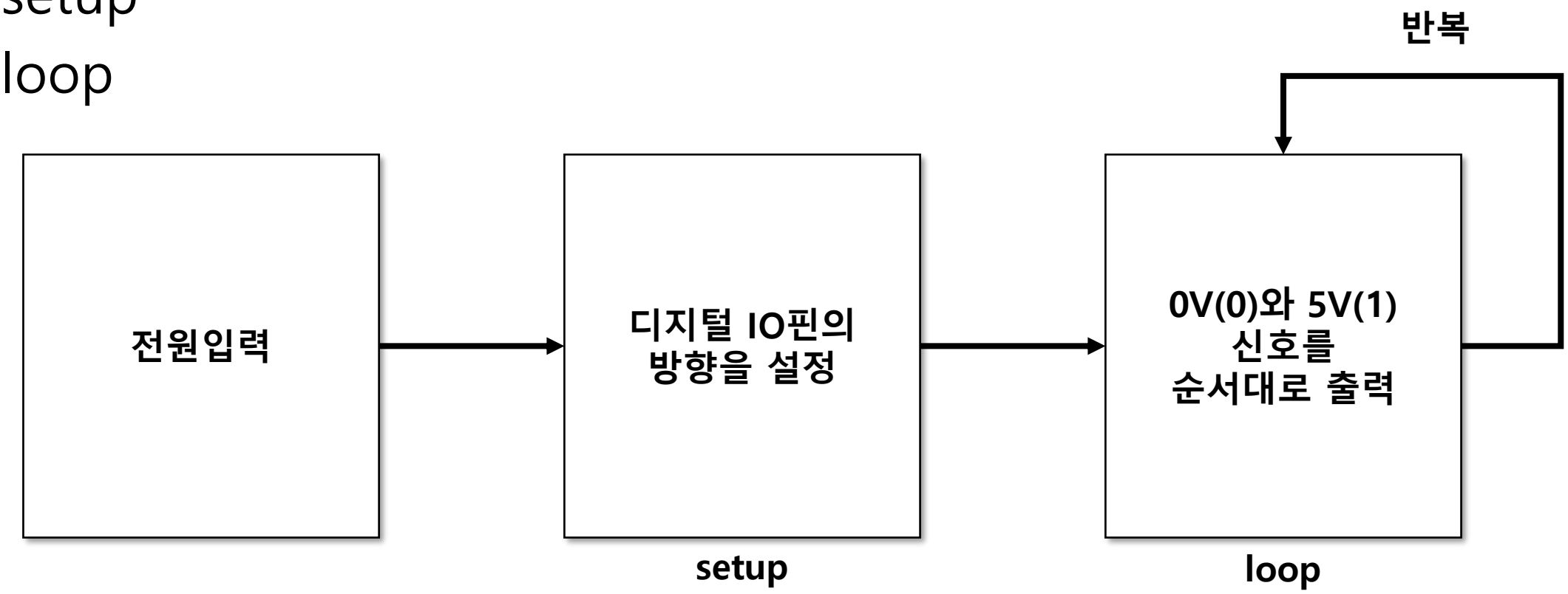
```
void setup()
{
  DDRB = B00000001 ;
  PORTB = B00000000 ;
}

void loop()
{
  PORTB = B00000001 ;
  delay(1000) ;
  PORTB = B00000000 ;
  delay(1000) ;
}
```



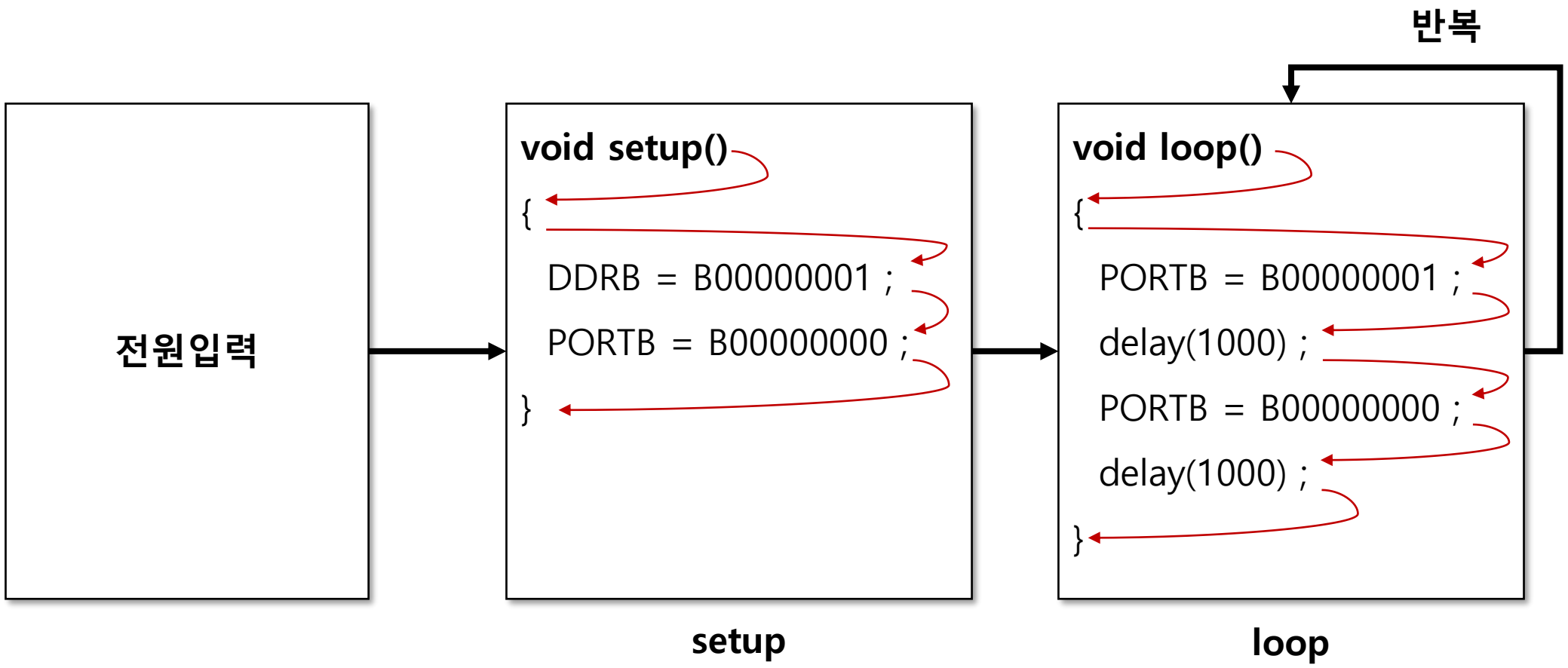


IO 포트 출력 실험

- setup
- loop



IO 포트 출력 실험


반복
전원입력
void setup()
{
DDRB = B00000001 ;
PORTB = B00000000 ;
}
setup
void loop()
{
PORTB = B00000001 ;
delay(1000) ;
PORTB = B00000000 ;
delay(1000) ;
}
loop

IO 포트 출력 실험

DDRB – The Port B Data Direction Register

Bit	7	6	5	4	3	2	1	0	
0x04 (0x24)	DDB7	DDB6	DDB5	DDB4	DDB3	DDB2	DDB1	DDB0	DDRB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

전원입력

```
void setup()
{
  DDRB = B00000001 ;
  PORTB = B00000000 ;
}
```

setup

```
{
  PORTB = B00000001 ;
  delay(1000) ;
  PORTB = B00000000 ;
```

loop

PORTB – The Port B Data Register

Bit	7	6	5	4	3	2	1	0	
0x05 (0x25)	PORTB7	PORTB6	PORTB5	PORTB4	PORTB3	PORTB2	PORTB1	PORTB0	PORTB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

IO 포트 출력 실험

반복

전원입력

```
void setup()
{
  DDRB = B00000001 ;
  PORTB = B00000000 ;
}
```

setup

```
void loop()
{
  PORTB = B00000001 ;
  delay(1000) ;
  PORTB = B00000000 ;
  delay(1000) ;
}
```

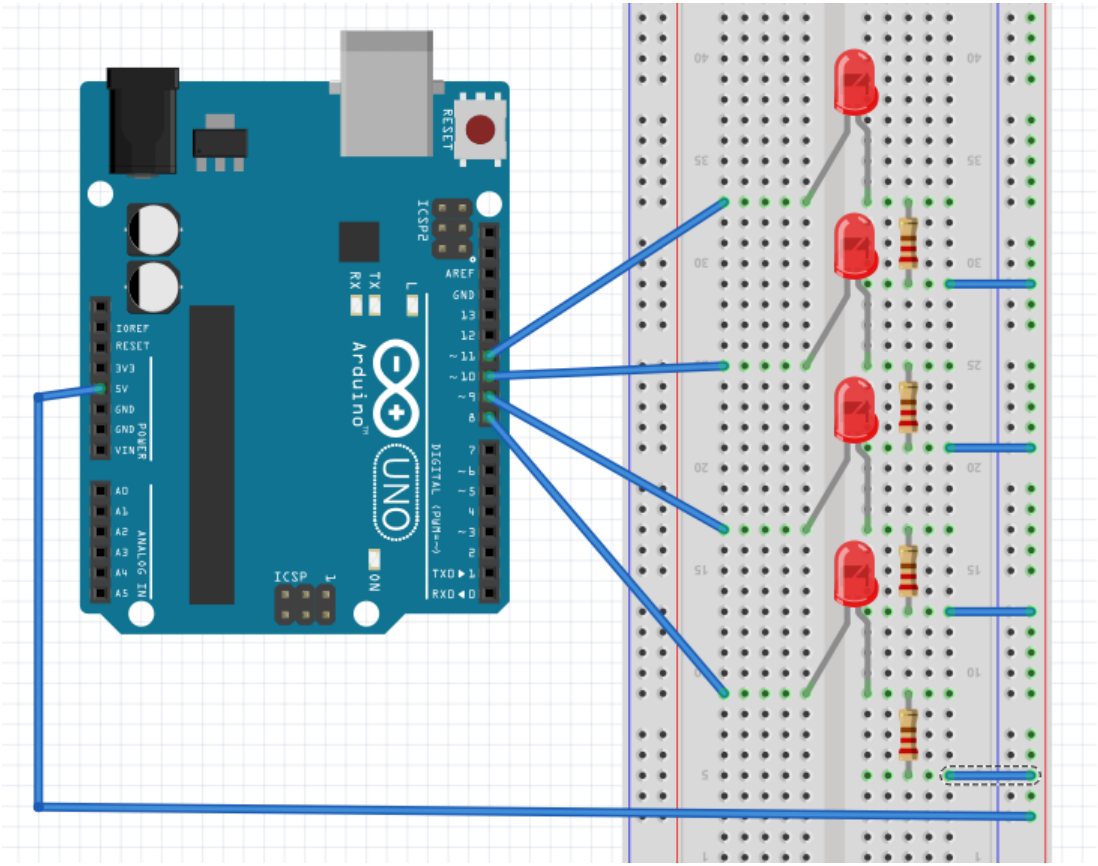
loop

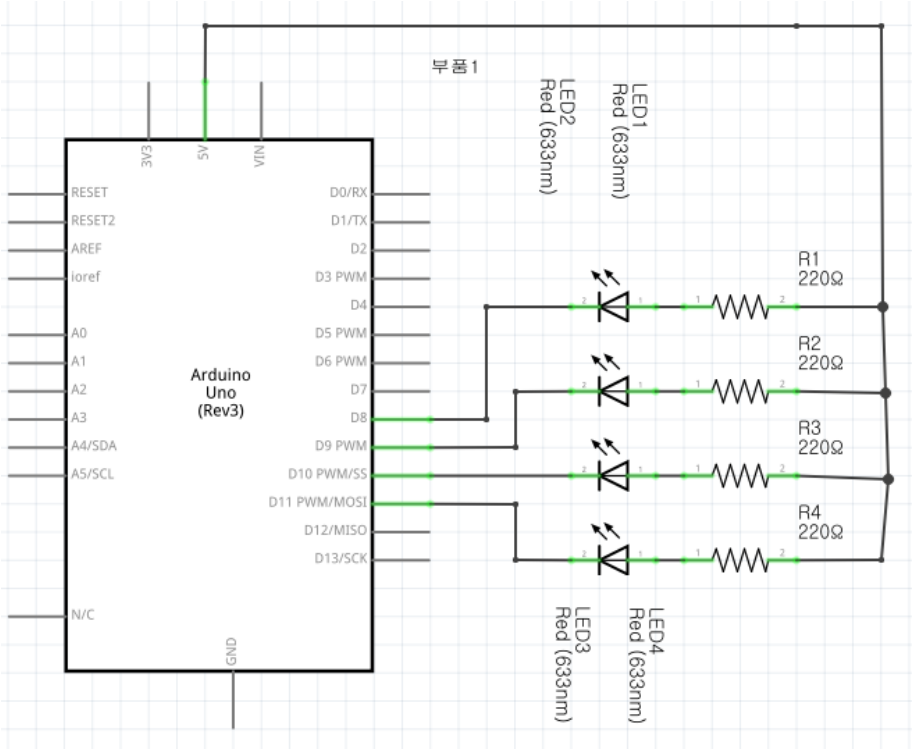
PORTB – The Port B Data Register

Bit	7	6	5	4	3	2	1	0	
0x05 (0x25)	PORTB7	PORTB6	PORTB5	PORTB4	PORTB3	PORTB2	PORTB1	PORTB0	PORTB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

IO 포트 출력 실험2

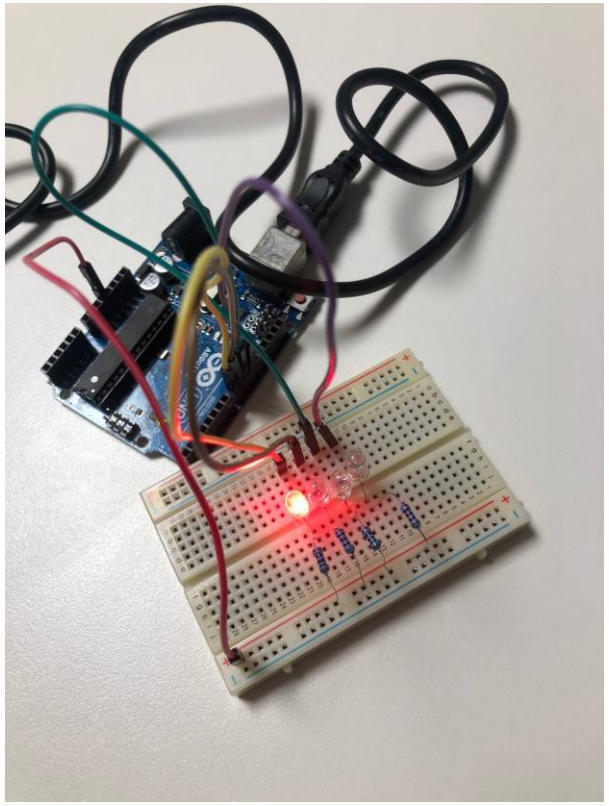
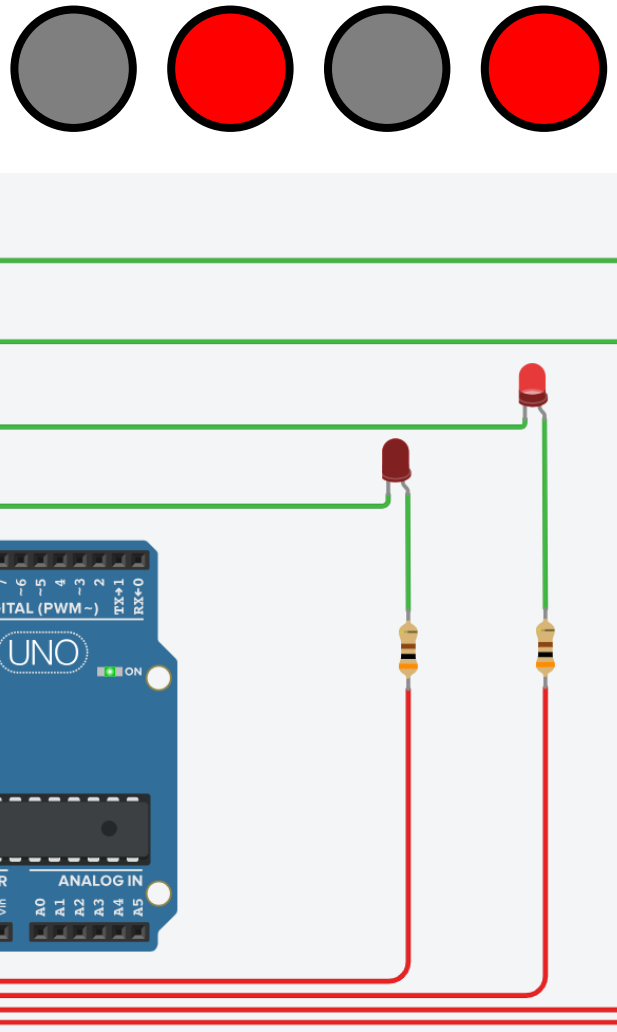
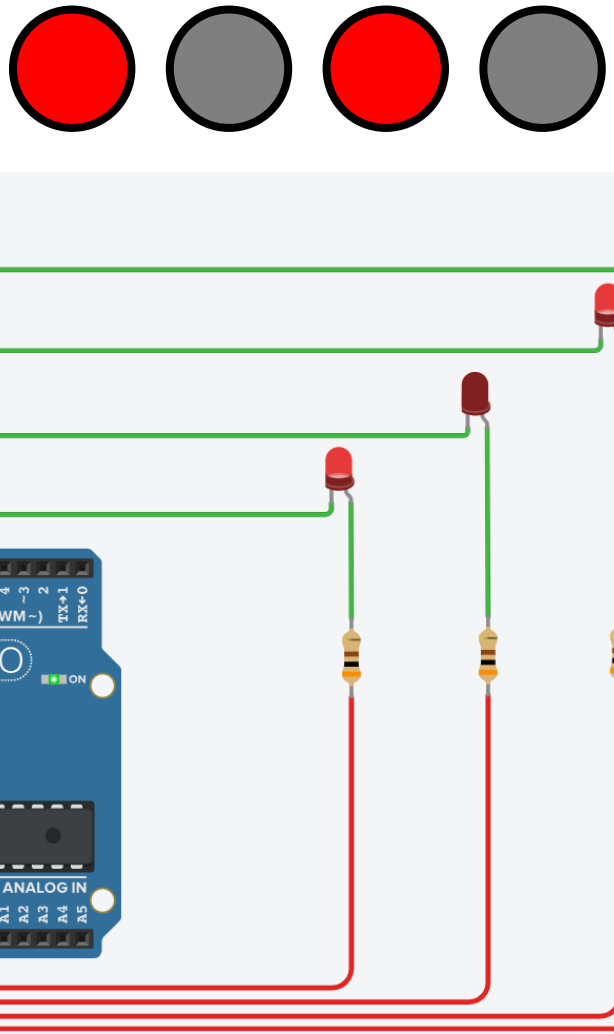
- 4개의 LED를 연결해서 포트를 제어 해보자.





IO 포트 출력 실험2

```
void setup()
{
  DDRB = B00001111 ;
  PORTB = B00000000 ;
}

void loop()
{
  PORTB = B00001111 ;
  delay(1000) ;
  PORTB = B00000000 ;
  delay(1000) ;
}
```



IO 포트 출력 실험3 - QUIZ